1 Methods

1.1 Core Idea

1.1.1 Setup

Consider a logistic regression, with missing features in the observation.
Let $x := (x^o, x^m) \in \mathbb{R}^d$. Then the logistic regression is defined as

$$p(y \mid x) := \sigma(w^T x), \qquad w \in \mathbb{R}^d.$$

Additionally we have a PC that models the density $p(x)$.

1.1.2 Goal

Compute the prediction in a probabilistically principled manner using

$$p(y \mid x^o) := \mathbb{E}_{p(x^m \mid x^o)} \left[\sigma \left(w^T x \right) \right].$$

If it were not for the non-linear sigmoid function σ, we would be able to calculate this in a smooth and decomposable PC, by pulling the Expectation -which is linear- into the leaf nodes. To tackle this we propose to use two tricks.

1.1.3 Trick 1: Pólya-Gamma

To replace the sigmoid function we use the PG-Sigmoid identity, shown in [1], that states:

$$\sigma(z) = \frac{1}{2} \exp\left(\frac{1}{2} z\right) \; \mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)} \left[\exp\left(-\frac{1}{2} \gamma z^2 \right) \right]$$

for $z \in \mathbb{R}$.

Intuitively, the identity writes the sigmoid as an average of bell-shaped curves $e^{z/2-\gamma z^2/2}$, one for every value of γ. In practice we replace the expectation over γ by a weighted sum over a few nodes γ_j with weights w_j (Section 1.2.3), so σ becomes a finite sum of weighted bells. Figure 1.1 shows how the sum of the bells with the largest weights approaches $\sigma(z)$: already four bells reproduce the sigmoid closely, and with all nodes the difference is no longer visible.

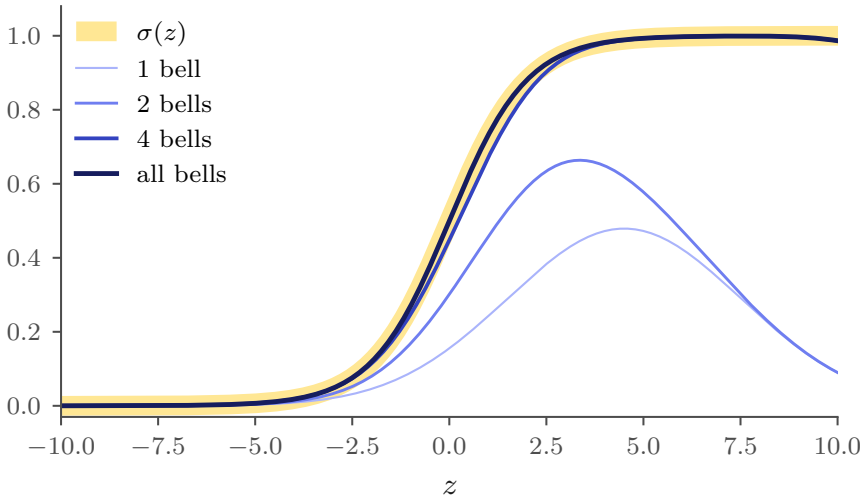


Figure 1.1: The Pólya-Gamma identity as a sum of bells. Partial sums of the weighted bells $\frac{1}{2}w_j e^{z/2-\gamma_j z^2/2}$, with nodes γ_j and weights w_j from Golub–Welsch (Section 1.2.3), largest weights first (light to dark), against $\sigma(z)$ (yellow). The first bell, the one with the largest weight, is decomposed further in Figure 1.2.

If we set $z := w^T x$, and insert into our expectation, we get:

$$\begin{aligned} p(y \mid x^o) &= \mathbb{E}_{p(x^m|x^o)}\left[\sigma(z)\right] \\ &= \mathbb{E}_{p(x^m|x^o)}\left[\frac{1}{2}\exp\left(\frac{1}{2}z\right)\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(-\frac{1}{2}\gamma z^2\right)\right]\right] \\ &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \end{aligned}$$

Now, we have a term without a sigmoid, but the x still appears in a quadratic form.

1.1.4 Trick 2: Hubbard-Stratonovich (HS)

To get rid of the quadratic form, we apply the Hubbard–Stratonovich identity [2, 3], that states:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \sqrt{\frac{1}{2\pi a}} \int_{-\infty}^{\infty} \exp\left(-\frac{u^2}{2a} - ihu\right) \mathrm{d}u, \tag{1.1}$$

Since $\frac{1}{\sqrt{2\pi a}} \exp\left(-\frac{u^2}{2a}\right)$ is the density of $\mathcal{N}(0, a)$, the integral is exactly the expectation of e^{-ihu} over $u \sim \mathcal{N}(0, a)$:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \mathbb{E}_{u\sim\mathcal{N}(0,\,a)}\left[e^{-ihu}\right]. \tag{1.2}$$

In plain words, it makes a quadratic term linear at the cost of introducing a helper variable that we need to integrate over.

Currently, in our term, x appears quadratically as well as linearly. To be able to apply HS, we complete the square (again using $z = w^T x$):

$$\begin{aligned}
\frac{1}{2}z - \frac{1}{2}\gamma z^2 &= -\frac{1}{2}\gamma\left(z^2 - \frac{z}{\gamma}\right) \\
&= -\frac{1}{2}\gamma\left(\left(z - \frac{1}{2\gamma}\right)^2 - \frac{1}{4\gamma^2}\right) \\
&= -\frac{1}{2}\gamma\left(z - \frac{1}{2\gamma}\right)^2 + \frac{1}{2}\gamma \cdot \frac{1}{4\gamma^2} \\
&= -\frac{1}{2}\gamma\left(z - \frac{1}{2\gamma}\right)^2 + \frac{1}{8\gamma}.
\end{aligned} \tag{1.3}$$

Now we apply HS with $a = \gamma$ and $h = z - \frac{1}{2\gamma}$:

$$\begin{aligned}
\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) &= \exp\left(\frac{1}{8\gamma}\right)\exp\left(-\frac{1}{2}\gamma\left(z - \frac{1}{2\gamma}\right)^2\right) \\
&= \exp\left(\frac{1}{8\gamma}\right)\sqrt{\frac{1}{2\pi\gamma}}\int_{-\infty}^{\infty}\exp\left(-\frac{u^2}{2\gamma} - i\left(z - \frac{1}{2\gamma}\right)u\right)\mathrm{d}u \\
&= \exp\left(\frac{1}{8\gamma}\right)\int_{-\infty}^{\infty}\underbrace{\frac{1}{\sqrt{2\pi\gamma}}\exp\left(-\frac{u^2}{2\gamma}\right)}_{\mathcal{N}(u;0,\gamma)}\exp\left(-i\left(z - \frac{1}{2\gamma}\right)u\right)\mathrm{d}u \\
&= \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-i\left(z - \frac{1}{2\gamma}\right)u\right)\right] \\
&= \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{iu}{2\gamma} - izu\right)\right] \qquad (1.4)
\end{aligned}$$

Since u and $-u$ are equally likely under $\mathcal{N}(0,\gamma)$, the imaginary parts cancel, and with $c := \frac{1}{2\gamma}$ the identity reads

$$\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) = \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\cos\left(u(z-c)\right)\right].$$

So every bell is an average of cosine waves centred at c, with frequencies $u \sim \mathcal{N}(0,\gamma)$. Again we replace the expectation by a weighted sum, this time over Gauss–Hermite nodes (Section 1.2.2). Figure 1.2 shows how the waves build up a bell: a single slow wave is far too wide, but as faster waves are added, they pile up where the bell is and cancel everywhere else.

Substituting back into $p(y \mid x^o)$, we get:

$$\begin{aligned}
p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \\
&= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{iu}{2\gamma} - izu\right)\right]\right] \\
&= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1+4iu}{8\gamma}\right)\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-izu\right)\right]\right]
\end{aligned}$$

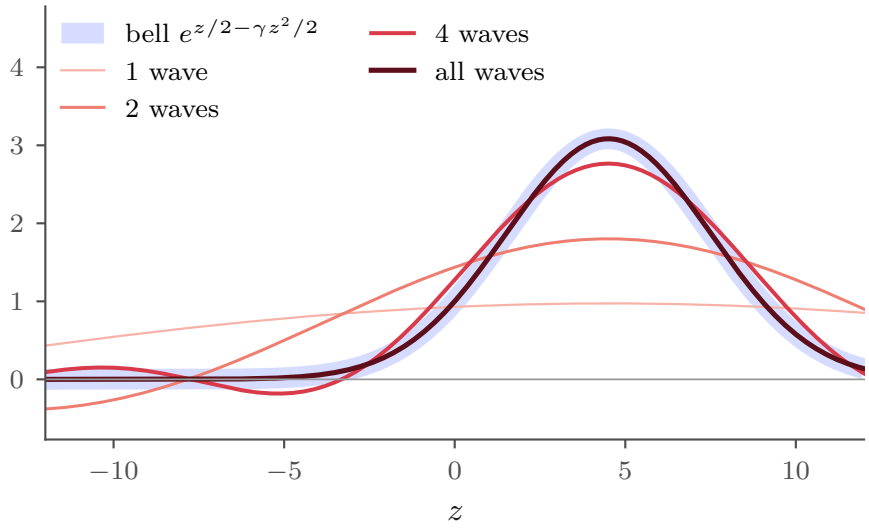


Figure 1.2: The Hubbard-Stratonovich identity as a sum of waves, for the bell with the largest weight in Figure 1.1. Partial sums of the weighted waves $e^{1/(8\gamma)}\frac{\omega_i}{\sqrt{\pi}}\cos(u_i(z-c))$, with Gauss–Hermite nodes $u_i = \sqrt{2\gamma}\,t_i$ and weights ω_i (Section 1.2.2), slowest first, where u_i and $-u_i$ together form one wave (light to dark), against the bell (blue).

1.1.5 Characteristic Function

The characteristic function of a random variable X is defined as $\phi_X(s) = \mathbb{E}\left[\exp(isX)\right]$. The innermost expectation is only over x^m, with x^o fixed, so we split $z = w^T x$ into its observed and missing parts:

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The x^o part is constant under $\mathbb{E}_{p(x^m|x^o)}$ and factors out:

$$\begin{aligned}\mathbb{E}_{p(x^m|x^o)}\left[\exp(-izu)\right] &= \mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-i(w_o^T x^o + w_m^T x^m)u\right)\right] \\ &= \exp\left(-iw_o^T x^o u\right)\cdot \mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iw_m^T x^m u\right)\right] \\ &= \exp\left(-iw_o^T x^o u\right)\cdot \phi_{x^m|x^o}(-uw_m)\end{aligned}$$

So the input to the characteristic function is $s := -uw_m$.

Figure 1.3 shows what this buys us. Let $p(z)$ denote the distribution of $z = w^T x$ under $p(x^m \mid x^o)$. For a fixed frequency u, the expectation of a wave under $p(z)$ is the signed area under the wave times $p(z)$. This area cannot be read off the density model directly, but it is exactly a value of the characteristic function of z:

$$\mathbb{E}_{p(z)}\left[\cos\left(u(z-c)\right)\right] = \mathrm{Re}\left[e^{-iuc}\,\phi_z(u)\right], \qquad \phi_z(-u) = \exp\left(-iw_o^T x^o u\right)\cdot\phi_{x^m|x^o}(-uw_m),$$

where the second equation is the one derived above. A slow wave hardly changes over $p(z)$ and keeps most of the area, while a fast wave oscillates within $p(z)$ and its positive and negative parts cancel; this is the decay of ϕ_z. Evaluating the characteristic function at all nodes u_i therefore gives the expectation of every wave, and this is the only place where the density model is used.

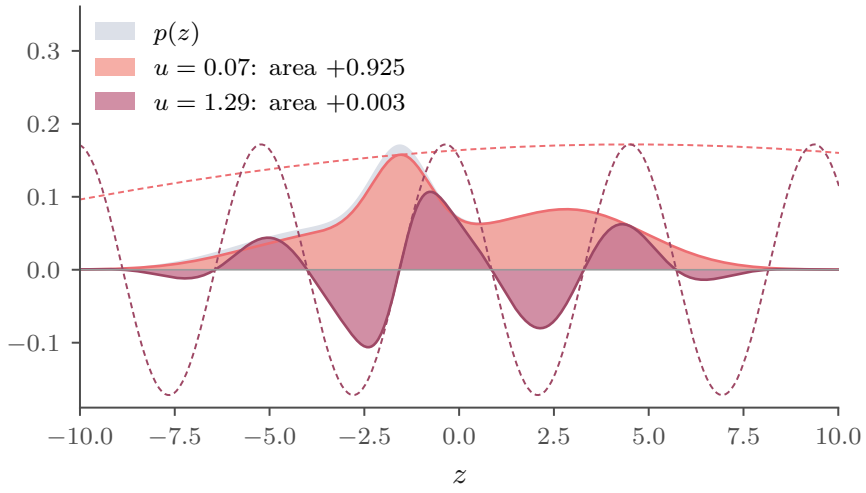


Figure 1.3: The expectation of a wave, for the bell of Figure 1.2 and the distribution $p(z)$ of $z = w^T x$ for a partially observed test point under a fitted GMM. A slow and a fast wave $\cos(u(z-c))$ (dashed, scaled) and their product with $p(z)$ (shaded). The signed shaded area is the expectation of the wave, which PCPG obtains as $\mathrm{Re}[e^{-iuc}\phi_z(u)]$ from the characteristic function.

1.1.6 Full Formula

Plugging the result of the previous subsection back into the outer expression for $p(y \mid x^o)$, we obtain:

$$\begin{aligned} p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1+4iu}{8\gamma}\right)\cdot\exp\left(-iw_o^T x^o u\right)\cdot\phi_{x^m|x^o}(-uw_m)\right] \\ &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}+\frac{iu}{2\gamma}-iw_o^T x^o u\right)\cdot\phi_{x^m|x^o}(-uw_m)\right] \\ &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}+iu\left(\frac{1}{2\gamma}-w_o^T x^o\right)\right)\cdot\phi_{x^m|x^o}(-uw_m)\right] \end{aligned}$$

We propose this as a final expression that we can tractably evaluate.

Figure 1.4 summarises the whole procedure. The expected prediction $p(y \mid x^o)$ is the area under $\sigma(z)\,p(z)$. Each bell contributes the area under the weighted bell times

$p(z)$, which PCPG computes from its waves and therefore from the characteristic function, and adding up these areas rebuilds the expected prediction. Since every bell is positive, the area only grows with each additional bell: the bell with the largest weight alone covers less than half of it, four bells come close, and with all nodes the estimate matches the exact value to four digits.

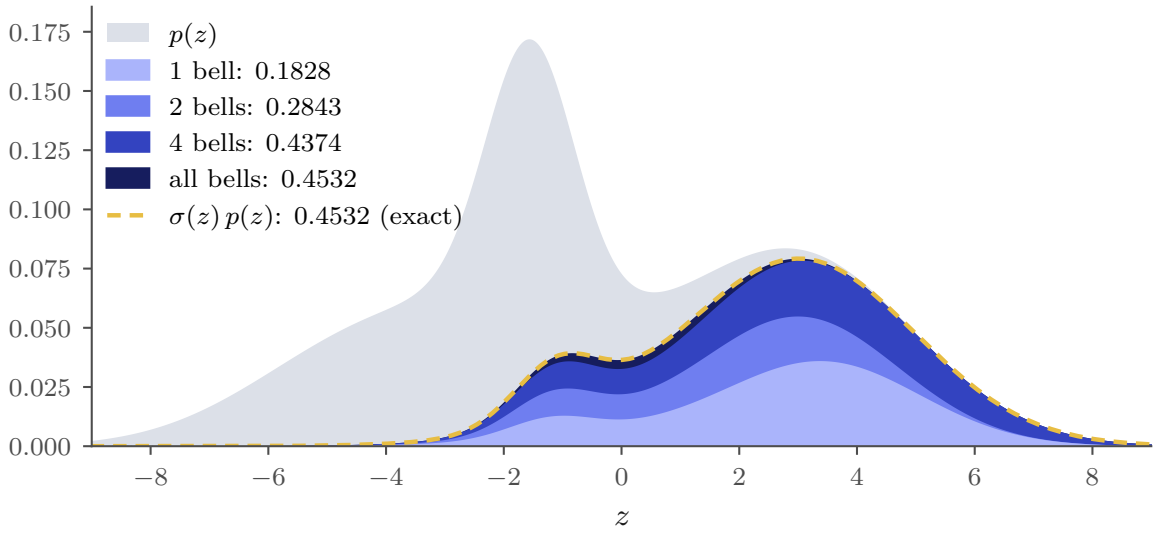


Figure 1.4: Summary of PCPG for the test point of Figure 1.3. The dashed line is $\sigma(z)\,p(z)$, whose area is the expected prediction $p(y \mid x^o)$. The shaded regions are the partial sums of the weighted bells times $p(z)$, largest weights first. The legend gives their areas as computed by PCPG, i.e. from the characteristic function through the sums of waves.

1.1.7 A Direct Route for Gaussian Mixtures

If the density $p(x)$ is a Gaussian mixture model (GMM), the goal can be computed directly, without either trick. We go through this in a few small steps.

Step 1: only one number matters. The prediction $\sigma(w^T x)$ does not look at the features one by one. It only looks at the single number

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The observed part $w_o^T x^o$ is a fixed number. The missing part $w_m^T x^m$ is uncertain, because x^m is uncertain. So z is a random number, and what we want is the average of $\sigma(z)$.

Step 2: average over z instead of over x^m. Many different values of x^m lead to the same z, and therefore to the same prediction. Instead of averaging σ over all possible x^m, we can group them by their value of z and average over z, weighted by how likely each value is:

$$\mathbb{E}_{p(x^m|x^o)}\left[\sigma(z)\right] = \mathbb{E}_{z\sim p(z)}\left[\sigma(z)\right].$$

No matter how many features are missing, this is an average over a single number. The only thing we need for it is the distribution $p(z)$.

Step 3: look at one mixture component at a time. A GMM describes the data as K bell-shaped clusters, the mixture components, numbered $k = 1, \ldots, K$. One can think of a data point as being generated in two steps: first a component k is picked, then x is drawn from that component's Gaussian, with means μ_{kj} and variances σ_{kj}^2 for each feature j. After seeing the observed features x^o, some components become more plausible than others; $\tilde{\pi}_k(x^o)$ denotes the probability that the point came from component k (see Section 1.2.1). Since the average over a mixture is the weighted average of the averages over its components, we can handle each component on its own:

$$p(y \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o)\, \mathbb{E}\left[\sigma(z) \mid k\right],$$

where $\mathbb{E}[\sigma(z) \mid k]$ is the average prediction assuming the point came from component k.

Step 4: within a component, z is Gaussian. Assume the point came from component k. Then each missing feature x_j is Gaussian with mean μ_{kj} and variance σ_{kj}^2, and the missing features are independent of each other. The number

$$z = w_o^T x^o + \sum_{j \text{ missing}} w_j x_j$$

is a fixed number plus a weighted sum of independent Gaussians, and such a sum is again Gaussian. Its mean and variance follow from two simple rules:

- the mean of a weighted sum is the weighted sum of the means, so each missing feature is replaced by its mean in component k:

$$m_k = w_o^T x^o + \sum_{j \text{ missing}} w_j\, \mu_{kj},$$

- multiplying x_j by w_j multiplies its variance by w_j^2, and variances of independent

variables add up. The observed part is fixed and adds no variance:

$$v_k = \sum_{j \text{ missing}} w_j^2 \, \sigma_{kj}^2.$$

We write this as $z \mid k \sim \mathcal{N}(m_k, v_k)$, read as "given that the point came from component k, z is normally distributed with mean m_k and variance v_k". For example, with $z = 0.5 + 2x_2 - x_3$, means $(\mu_{k2}, \mu_{k3}) = (1, 0)$ and variances $(\sigma_{k2}^2, \sigma_{k3}^2) = (1, 4)$, we get $m_k = 0.5 + 2 \cdot 1 - 1 \cdot 0 = 2.5$ and $v_k = 2^2 \cdot 1 + (-1)^2 \cdot 4 = 8$.

Together with Step 2, each component reduces to the average of the sigmoid under a one-dimensional normal distribution:

$$p(y \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \mathbb{E}_{z \sim \mathcal{N}(m_k, v_k)} \left[\sigma(z) \right].$$

Step 5: compute each one-dimensional average. The average of the sigmoid under a normal distribution has no closed form, but it is exactly the kind of integral Gauss–Hermite quadrature is made for (Section 1.2.2): evaluate σ at a few fixed points around the mean m_k, spread according to $\sqrt{v_k}$, and take a weighted sum. This gives

$$p(y \mid x^o) \approx \frac{1}{\sqrt{\pi}} \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \sum_{i=1}^{N} \omega_i \, \sigma\Big(m_k + \sqrt{2 v_k} \, t_i \Big).$$

Why this does not work for a general PC. The route above needs two things: that we can go through the components one by one (Step 3), and that z is Gaussian within each of them (Step 4). A PC is also a mixture, but the number of its components grows exponentially with its size, so we cannot go through them one by one. And with non-Gaussian leaves, z is not Gaussian, so its distribution is not known. What a PC can compute is the average of a function that splits into one factor per feature, by pushing the average down to the leaves. The sigmoid does not split this way. The exponential $e^{-izu} = \prod_j e^{-iw_j x_j u}$ does, and its average is the characteristic function. This is why, for general PCs, we rewrite the sigmoid in terms of such exponentials, which leads to the full formula above.

1.2 Implementation

Two ingredients are still abstract in the formula above: the conditional characteristic function $\phi_{x^m \mid x^o}$, which depends on the density model, and the inner $\mathcal{N}(0, \gamma)$ expectation, which needs to be approximated. This section makes both concrete.

1.2.1 Conditional CF for a GMM

For a Gaussian mixture model $p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$ with diagonal covariances $\Sigma_k = \mathrm{diag}(\sigma_k^2)$, the conditional density $p(x^m \mid x^o)$ is again a mixture, with the same component shapes restricted to the missing coordinates and reweighted by the posterior responsibility of each component given x^o:

$$\tilde{\pi}_k(x^o) = \frac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}.$$

Since the characteristic function of $\mathcal{N}(\mu, \Sigma)$ at s is $\exp\left(is^T\mu - \frac{1}{2}s^T\Sigma s\right)$, and the CF of a mixture is the same convex combination of component CFs, we get a closed-form expression:

$$\phi_{x^m|x^o}(s) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \exp\left(is^T\mu_k^m - \tfrac{1}{2}s^T\Sigma_k^m s\right),$$

where μ_k^m, Σ_k^m are the component means and covariances restricted to the missing coordinates.

1.2.2 Gauss–Hermite Quadrature

The inner Gaussian expectation is amenable to Gauss–Hermite quadrature, with deterministic nodes $\{t_i\}_{i=1}^N$ and weights $\{\omega_i\}_{i=1}^N$ such that

$$\int_{-\infty}^{\infty} e^{-t^2} g(t)\,\mathrm{d}t \approx \sum_{i=1}^{N} \omega_i \, g(t_i)$$

To use this we need to bring the $\mathcal{N}(0, \gamma)$ expectation into the form $\int e^{-t^2} g(t)\,\mathrm{d}t$. Writing the expectation out as an integral against the Gaussian density gives

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}[f(u)] = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi\gamma}} \exp\left(-\frac{u^2}{2\gamma}\right) f(u)\,\mathrm{d}u.$$

Apply the substitution $u = \sqrt{2\gamma}\,t$. It changes three pieces of the integrand:

- the differential: $\mathrm{d}u = \sqrt{2\gamma}\,\mathrm{d}t$,
- the Gaussian exponent: $\dfrac{u^2}{2\gamma} = \dfrac{(\sqrt{2\gamma}\,t)^2}{2\gamma} = \dfrac{2\gamma\,t^2}{2\gamma} = t^2$,
- the argument of f: $f(u) = f(\sqrt{2\gamma}\,t)$,

and the integration limits are unchanged since $\sqrt{2\gamma} > 0$. Plugging all three substitutions into the integral:

$$\begin{aligned}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}[f(u)] &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi\gamma}}\, e^{-t^2} f\Big(\sqrt{2\gamma}\, t\Big)\, \sqrt{2\gamma}\, \mathrm{d}t \\ &= \frac{\sqrt{2\gamma}}{\sqrt{2\pi\gamma}} \int_{-\infty}^{\infty} e^{-t^2} f\Big(\sqrt{2\gamma}\, t\Big)\, \mathrm{d}t.\end{aligned}$$

The constant prefactor simplifies to

$$\frac{\sqrt{2\gamma}}{\sqrt{2\pi\gamma}} = \sqrt{\frac{2\gamma}{2\pi\gamma}} = \sqrt{\frac{1}{\pi}} = \frac{1}{\sqrt{\pi}},$$

so the Gaussian expectation becomes

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}[f(u)] = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} e^{-t^2} f\Big(\sqrt{2\gamma}\, t\Big)\, \mathrm{d}t.$$

Which we then can apply by

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}[f(u)] \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^{N} \omega_i\, f\Big(\sqrt{2\gamma}\, t_i\Big)\,.$$

Plugging this into the inner expectation of our final formula, with

$$f(u) = \exp\left(\frac{1}{8\gamma} + iu\left(\frac{1}{2\gamma} - w_o^T x^o\right)\right) \phi_{x^m|x^o}(-uw_m)$$

gives

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)} \left[\sum_{i=1}^{N} \omega_i\, \exp\left(\frac{1}{8\gamma} + i\sqrt{2\gamma}\, t_i \left(\frac{1}{2\gamma} - w_o^T x^o\right)\right) \phi_{x^m|x^o}\Big(-\sqrt{2\gamma}\, t_i\, w_m\Big)\right]$$

1.2.3 Approximating the PG integral

We also want don't want to sample the outer expectation over $\gamma \sim \mathrm{PG}(1,0)$ and construct a set of fixed nodes and weights that "approximate" the PG(1,0) measure best. From what we found, there are multiple ways to do this.

All of them replace the expectation by a weighted sum, $\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}[f(\gamma)] \approx \sum_{j=1}^{n} w_j f(\gamma_j)$, with nodes $\gamma_j > 0$ and weights $w_j \geq 0$ summing to one. To see what such a rule has to achieve, note that the function of γ in the PG identity is $e^{-\gamma z^2/2}$, and its expectation

is the Laplace transform of PG$(1,0)$, $\mathcal{L}(t) := \mathbb{E}[e^{-t\gamma}] = \operatorname{sech}(\sqrt{t/2})$, evaluated at $t = z^2/2$. The PG identity can therefore be written as

$$\sigma(z) = \tfrac{1}{2}\, e^{z/2}\, \mathcal{L}\Big(\tfrac{z^2}{2}\Big), \qquad \hat{\sigma}(z) = \tfrac{1}{2}\, e^{z/2} \sum_{j=1}^{n} w_j\, e^{-\gamma_j z^2/2}, \tag{1.5}$$

where $\hat{\sigma}$ is what a rule with n nodes produces. Every rule below is a different way of making $\sum_j w_j e^{-\gamma_j t}$ close to $\mathcal{L}(t)$. For large $|z|$ this is hard: $e^{-\gamma z^2/2}$ is then only non-negligible for very small γ, so the rules differ mainly in how well they cover the region near $\gamma = 0$.

Golub–Welsch construction

One is the *Golub–Welsch algorithm* [4], which produces n-point quadrature nodes and weights from the first $2n$ moments of any positive measure.

The Laplace transform of PG$(1,0)$ is known in closed form:

$$\mathcal{L}(t) := \mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\Big[e^{-t\gamma}\Big] = \operatorname{sech}\Big(\sqrt{\tfrac{t}{2}}\Big) = \frac{1}{\cosh\Big(\sqrt{t/2}\Big)}.$$

The raw moments follow from derivatives at zero:

$$\mu_k := \mathbb{E}[\gamma^k] = (-1)^k\, \mathcal{L}^{(k)}(0),$$

which can be evaluated numerically to arbitrary precision.

Given $\mu_0, \ldots, \mu_{2n-1}$, the algorithm goes as follows. Form the $n \times n$ Hankel moment matrices

$$H_{ij} = \mu_{i+j}, \qquad H'_{ij} = \mu_{i+j+1}, \quad i, j = 0, \ldots, n-1.$$

Compute the Cholesky factor $H = R^\top R$ and the Jacobi (tridiagonal) matrix

$$J = R^{-\top} H' R^{-1}, \quad \text{symmetrised as } J \leftarrow \tfrac{1}{2}(J + J^\top).$$

Eigendecompose $J = V \Lambda V^\top$. Then the quadrature nodes and weights are

$$\gamma_j = \lambda_j, \qquad w_j = \mu_0\, v_{0j}^2, \quad j = 1, \ldots, n,$$

where λ_j are the eigenvalues and v_{0j} is the first component of the j-th eigenvector. The resulting rule integrates polynomials in γ exactly up to degree $2n-1$.

The Hankel matrix is extremely ill-conditioned, so the whole computation is done in arbitrary precision (200 digits). Even then, the rule is only reliable up to $n \approx 19$.

Log-trapezoid rule

Substituting $\gamma = e^s$ turns the expectation into

$$\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}[f(\gamma)] = \int_{-\infty}^{\infty} f(e^s)\, p(e^s)\, e^s \,\mathrm{d}s,$$

where p is the density of $\mathrm{PG}(1,0)$. Applying the trapezoid rule with n equally spaced nodes s_j and spacing h gives nodes $\gamma_j = e^{s_j}$ and weights $w_j \propto p(\gamma_j)\,\gamma_j\,h$. The density has no closed form, but two alternating series, one converging fast for small and one for large γ, evaluate it to machine precision. Since p decays like $e^{-1/(8\gamma)}$ towards 0 and like $e^{-\pi^2\gamma/2}$ towards infinity, the range $\gamma \in [10^{-3}, 10]$ covers all of the mass. Equal spacing in $\log\gamma$ places many nodes at small γ, exactly where $e^{-\gamma z^2/2}$ lives for large $|z|$, and for smooth integrands that vanish at both ends the trapezoid rule converges exponentially in n. Unlike Golub–Welsch, it works for any n.

Quantile rule

The simplest deterministic rule places the nodes at the quantiles of $\mathrm{PG}(1,0)$,

$$\gamma_j = F^{-1}\!\left(\frac{j-\frac{1}{2}}{n}\right), \qquad w_j = \frac{1}{n},$$

with F the CDF of $\mathrm{PG}(1,0)$, again computed from a series. This is the midpoint rule in probability space, i.e. one-dimensional quasi-Monte Carlo: the same equal weights as MC, but with the nodes spread evenly instead of drawn at random. Its error decays roughly like $1/n^2$. We include it mainly as a baseline between MC and the more elaborate rules.

Nonnegative least-squares construction

The Golub–Welsch rule matches the *moments* of $\mathrm{PG}(1,0)$, i.e. the Taylor coefficients of $\mathcal{L}$ at $t = 0$, so its accuracy is concentrated near $z = 0$ and decays as $|z|$ grows. An alternative is to match the Laplace transform directly across the range of z.

Fix M candidate support locations $\{\gamma_j\}_{j=1}^{M}$ log-spaced in $[\gamma_{\min}, \gamma_{\max}]$, and $R \gg M$ fitting points $\{z_r\}_{r=1}^{R}$ linearly spaced in $[0, z_{\max}]$. Define

$$A_{rj} = e^{-z_r\gamma_j}, \qquad b_r = \mathcal{L}(z_r) = \operatorname{sech}\!\left(\sqrt{z_r/2}\right),$$

and solve the nonnegative least-squares problem

$$\min_{w \geq 0} \ \|Aw - b\|_2^2.$$

The solution w is then normalized to sum to one, and support points with $w_j < \varepsilon$ are eliminated. The remaining pairs (γ_j, w_j) serve as quadrature nodes and weights for the outer PG integral. NNLS solutions are sparse, so only around a dozen of the M candidates are used, and the number of nodes is not chosen directly.

The fit only constrains $\mathcal{L}$ for $t \leq z_{\max}$, which corresponds to $|z| \leq \sqrt{2z_{\max}}$. If the candidate grid reaches very small γ (e.g. 10^{-5}), the fit puts weight on a node whose exponential is almost constant on that range, because this helps to match the slowly decaying tail of $\mathcal{L}$. Outside the fitted range this constant does not decay, and through the factor $e^{z/2}$ in (1.5) the estimate of $\sigma(z)$ grows without bound. We therefore start the grid at $\gamma_{\min} = 10^{-2}$.

Fully deterministic formula.

Combining PG nodes $(\gamma_j, w_j)_{j=1}^M$ with Gauss–Hermite nodes $(t_i, \omega_i)_{i=1}^N$ yields the deterministic approximation

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \sum_{j=1}^{M} w_j \sum_{i=1}^{N} \omega_i \, \exp\!\left(\frac{1}{8\gamma_j} + i\sqrt{2\gamma_j}\, t_i \left(\frac{1}{2\gamma_j} - w_o^T x^o\right)\right) \phi_{x^m|x^o}\!\left(-\sqrt{2\gamma_j}\, t_i \, w_m\right).$$

1.2.4 Sign Flip

Why large positive z is hard

In (1.5), any error ε a rule makes in approximating $\mathcal{L}(z^2/2)$ enters the estimate of $\sigma(z)$ multiplied by $\frac{1}{2}e^{z/2}$. The nodes only see z^2, so this error is the same for z and $-z$, but its effect is not:

$$\left|\hat{\sigma}(z) - \sigma(z)\right| = \tfrac{1}{2}\, e^{z/2}\, \varepsilon(z^2/2).$$

For $z < 0$ the factor shrinks the error, for $z > 0$ it amplifies it exponentially. Staying at $\sigma(z) \approx 1$ for large positive z requires the sum of exponentials to reproduce the tail $\mathcal{L}(z^2/2) \approx 2e^{-z/2}$ almost exactly, which only rules with nodes very close to $\gamma = 0$ can do. Rules without such nodes (Golub–Welsch, quantile) have a tail that is too small, so $\hat{\sigma}(z)$ falls back to 0; a rule with too much weight near 0 (NNLS with $\gamma_{\min} = 10^{-5}$) has a tail that is too heavy, so $\hat{\sigma}(z)$ blows up. Only the log-trapezoid rule stays accurate without further help, and only with enough nodes.

The flip

The sigmoid is point symmetric around $(0, \frac{1}{2})$:

$$\sigma(-z) = \frac{1}{1+e^{z}} = \frac{e^{-z}}{e^{-z}+1} = 1 - \sigma(z).$$

For $z > 0$ we can therefore compute $1 - \hat{\sigma}(-z)$ instead of $\hat{\sigma}(z)$. Both use the same nodes and the same approximation of $\mathcal{L}(z^2/2)$; only the factor multiplying its error changes from $e^{z/2}$ to $e^{-z/2}$. The error at z becomes the error at $-z$, which is small for every rule. In the one-dimensional identity this reduces the maximum error over $|z| \le 30$ of Golub–Welsch with 19 nodes from about 1 to $6.5 \cdot 10^{-7}$, and of the quantile rule with 15 nodes from about 1 to $4 \cdot 10^{-4}$.

The identity holds for the whole expectation as well,

$$\mathbb{E}\Big[\sigma(w^T x) \mid x^o\Big] = 1 - \mathbb{E}\Big[\sigma(-w^T x) \mid x^o\Big],$$

so the estimator can run with $-w$ and return one minus the result. Since $z = w^T x$ is random, the flip is decided by the sign of its conditional mean $\mathbb{E}[w^T x \mid x^o] = w_o^T x^o + w_m^T \mathbb{E}[x^m \mid x^o]$, which is the expectation of a linear function and therefore cheap for a GMM as well as for a PC. Part of the distribution of z can still lie on the positive side, but close to 0, where the amplification is moderate. For a GMM, where z is Gaussian within each component, the flip can also be applied per component.

Fitting the nodes to the sigmoid

With the flip, the rule is only evaluated for z around and below 0, and the error that matters is $\frac{1}{2} e^{z/2} \varepsilon(z^2/2)$ on that side. None of the rules above optimise this quantity: Golub–Welsch matches moments, the log-trapezoid and quantile rules discretise the PG distribution, and NNLS matches $\mathcal{L}$ with equal weight on all $t \le z_{\max}$.

Since the rule enters the estimator linearly, the estimator computes exactly $\mathbb{E}[\hat{\sigma}(z)]$, and its error is the error of $\hat{\sigma}$ averaged over the distribution of z. Nothing in the rest of the method requires the nodes to represent PG$(1, 0)$: HS and the characteristic function only need $\gamma_j > 0$, and the weights only need to be nonnegative. We can therefore choose the nodes and weights to approximate the sigmoid itself,

$$\min_{w \ge 0} \sum_r \left(\tfrac{1}{2}\, e^{z_r/2} \sum_{j=1}^{M} w_j\, e^{-\gamma_j z_r^2/2} - \sigma(z_r) \right)^2,$$

over a log-spaced candidate grid γ_j and fitting points $z_r \in [-30, 5]$, again solved with NNLS. This is the NNLS fit of $\mathcal{L}$ from above, but with the errors weighted by $\frac{1}{2} e^{z/2}$,

i.e. by exactly how much they affect the sigmoid. The margin up to $z = 5$ covers the part of the distribution of z that remains on the positive side after the flip. The grid starts at $\gamma_{\min} = 0.02$, so that $e^{1/(8\gamma)}$ in the inner integral stays below about 500. The resulting rule has about 12 nodes and a maximum error of about 10^{-8} on $[-30, 5]$, roughly a hundred times smaller than Golub–Welsch with 15 nodes.

This changes the view of the outer integral: the method approximates the sigmoid by functions of the form $\frac{1}{2}e^{z/2}e^{-\gamma z^2/2}$, whose expectations are tractable, and the PG identity is one particular, exact but infinite, choice of how to combine them. A finite rule only has to combine them well, not to represent the PG distribution. The only variants this does not apply to are the ones that sample γ from $\mathrm{PG}(1, 0)$, since a fitted rule cannot be sampled from.

1.2.5 Algorithms

Putting the pieces together, we can define three variants of our method, depending on how we want to approximate the integrals. A full Monte Carlo version samples both γ_j and u_i; a Gauss–Hermite version samples only γ_j and uses quadrature for u_i; and a fully deterministic version uses quadrature for both.

Full Monte Carlo

Below is an algorithm for the simplest - fully Monte Carlo - variant.

Algorithm 1 PCPG: expected sigmoid under a GMM, full Monte Carlo

Input: LR weights w, partial observation x^o, GMM parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$, # PG samples M, # Gaussian samples N

Split $w = (w_o, w_m)$ and $\mu_k = (\mu_k^o, \mu_k^m)$, $\Sigma_k = (\Sigma_k^o, \Sigma_k^m)$ by the observed/missing mask

$\tilde{\pi}_k \leftarrow \dfrac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_j \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}$ for $k = 1, \ldots, K$

$c \leftarrow w_o^T x^o$

$S \leftarrow 0$

for $j = 1, \ldots, M$ **do**

  Sample $\gamma_j \sim \mathrm{PG}(1, 0)$

  for $i = 1, \ldots, N$ **do**

    Sample $u \sim \mathcal{N}(0, \gamma_j)$

    $s \leftarrow -u\, w_m$

    $\phi \leftarrow \sum_{k=1}^K \tilde{\pi}_k \exp\!\left(is^T \mu_k^m - \frac{1}{2} s^T \Sigma_k^m s\right)$

    $\text{factor} \leftarrow \exp\left(\dfrac{1}{8\gamma_j} + iu\left(\dfrac{1}{2\gamma_j} - c\right)\right)$

    $S \leftarrow S + \text{factor} \cdot \phi$

  end for

end for

return $\dfrac{1}{2\,M\,N} \cdot \mathrm{Re}(S)$

Unified version

Since all variant share the same core structure, we don't include separate algorithms for all, but but instead give a unified version that takes the outer and inner nodes/weights as input. Table 1.1 summarises the differences; Algorithm 2 gives the unified procedure.

Variant	**Outer** (γ_j, w_j)	**Inner** $(u_i, \omega_i \mid \gamma_j)$	**Norm** Z
PCPG-MC	M samples PG(1, 0), w_j=1	N samples $\mathcal{N}(0, \gamma_j)$, ω_i=1	$\frac{1}{2MN}$
PCPG-GH	M samples PG(1, 0), w_j=1	Hermite: u_i=$\sqrt{2\gamma_j}\, t_i$	$\frac{1}{2\sqrt{\pi}M}$
PCPG-Quad	GW nodes/weights	Hermite: u_i=$\sqrt{2\gamma_j}\, t_i$	$\frac{1}{2\sqrt{\pi}}$

Table 1.1: The three PCPG variants. GW = Golub–Welsch (§1.2.3); Hermite nodes (t_i, ω_i) from `hermgauss`.

Algorithm 2 PCPG (unified)

Input: LR weights w, partial observation x^o, GMM parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$, outer iterates $(\gamma_j, w_j)_{j=1}^M$, inner iterates $(u_i, \omega_i)_{i=1}^N$ (per γ_j), normalisation Z

Split $w = (w_o, w_m)$ by the observed/missing mask

$\tilde{\pi}_k \leftarrow \dfrac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_j \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}$ for $k = 1, \ldots, K$

$c \leftarrow w_o^T x^o, \quad S \leftarrow 0$

for $j = 1, \ldots, M$ **do**

  for $i = 1, \ldots, N$ **do**

    $s \leftarrow -u_i \, w_m$

    $\phi \leftarrow \sum_k \tilde{\pi}_k \exp\!\left(i s^T \mu_k^m - \tfrac{1}{2} s^T \Sigma_k^m s\right)$

    $\text{factor} \leftarrow \exp\!\left(\dfrac{1}{8\gamma_j} + i u_i \left(\dfrac{1}{2\gamma_j} - c\right)\right)$

    $S \leftarrow S + w_j \, \omega_i \, \text{factor} \cdot \phi$

  end for

end for

return $Z \cdot \mathrm{Re}(S)$

Of course one could also add a forth variant that uses quadrature for the PG integral but sampling for the inner Gaussian, but we don't include it here since it doesn't perform well in practice. The variance of the inner Gaussian sampling is very high which makes the quadrature perform much better.

2 Discussion

2.1 Variance Analysis

2.1.1 Conditional Sampling MC Estimator

The MC estimator draws N samples $x^{m(i)} \sim p(x^m \mid x^o)$ and returns

$$\widehat{p}_{\mathrm{MC}}(N) = \frac{1}{N}\sum_{i=1}^{N} \sigma\Big(w^T x^{(i)}\Big)\,.$$

Since $\sigma(\cdot) \in (0,1)$, the per-sample variance $\sigma^2_{\mathrm{MC}} := \mathrm{Var}_{p(x^m|x^o)}\Big[\sigma(w^T x)\Big]$ is bounded by $1/4$, giving

$$\mathrm{Var}[\widehat{p}_{\mathrm{MC}}(N)] = \frac{\sigma^2_{\mathrm{MC}}}{N} \;\le\; \frac{1}{4N}.$$

The $1/N$ factor follows from the standard variance-of-a-sample-mean identity: since the draws are i.i.d., the variance of their average is the per-sample variance divided by N. The bound $\sigma^2_{\mathrm{MC}} \le 1/4$ holds because $\sigma(\cdot) \in (0,1)$: for any $[0,1]$-valued random variable Y,

$$\mathrm{Var}(Y) = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2 \le \mathbb{E}[Y] - \mathbb{E}[Y]^2 = \mathbb{E}[Y](1-\mathbb{E}[Y]) \le \tfrac{1}{4},$$

where the first inequality uses $Y^2 \le Y$ on $[0,1]$ and the second uses $p(1-p) \le 1/4$ for all $p \in [0,1]$, with equality at $p = 1/2$.

2.1.2 PCPG-MC Estimator

The PCPG-MC estimator with M outer (PG) and L inner (Gaussian) samples returns

$$\widehat{p}_{\mathrm{PCPG}}(M,L) = \frac{1}{2ML}\sum_{j=1}^{M}\sum_{i=1}^{L} \mathrm{Re}[h(\gamma_j, u_{ij})]\,,$$

where $h(\gamma,u) := \exp\Big(\frac{1}{8\gamma} + iu\Big(\frac{1}{2\gamma} - c\Big)\Big)\cdot\phi_{x^m|x^o}(-uw_m)$, $c = w_o^T x^o$, and $u_{ij} \sim \mathcal{N}(0,\gamma_j)$.

Output range

A single sample (M=L=1) returns $\frac{1}{2}\mathrm{Re}[h(\gamma, u)]$.

The characteristic function $\phi(s) = \mathbb{E}[e^{isx}]$ satisfies $|\phi(s)| \leq 1$ for all s.

Since $|h(\gamma, u)| = e^{1/(8\gamma)} \cdot |\phi(-uw_m)| \leq e^{1/(8\gamma)}$, the value lies in $\left[-\frac{1}{2}e^{1/(8\gamma)}, +\frac{1}{2}e^{1/(8\gamma)}\right]$.

Because $\gamma \sim \mathrm{PG}(1, 0)$ can be arbitrarily small, $e^{1/(8\gamma)}$ is unbounded, so individual PCPG samples are not confined to $[0, 1]$ and can be negative or exceed 1.

By contrast, every individual MC sample lies in $(0, 1)$.

Variance

The PCPG-MC estimator has two nested levels of randomness.

1. **Outer level**: M independent draws $\gamma_j \sim \mathrm{PG}(1, 0)$.
2. **Inner level**: for each j, L independent draws $u_{ij} \sim \mathcal{N}(0, \gamma_j)$.

Let $Z = \frac{1}{2}\mathrm{Re}[h(\gamma, u)]$ denote a single "raw" sample. The estimator is the mean of ML such values, grouped so that each outer sample γ_j drives a block of L inner values.

Step 1: variance within one outer block. Fix γ_j. The L inner samples u_{ij} are i.i.d. $\mathcal{N}(0, \gamma_j)$, so the block average $\bar{Z}_j := \frac{1}{L}\sum_{i=1}^{L} Z_{ij}$ has conditional variance $\sigma^2_{A,\gamma_j}/L$, where $\sigma^2_{A,\gamma} := \mathrm{Var}_{u|\gamma}[Z \mid \gamma]$.

Step 2: law of total variance for $\bar{Z}_j$. The M blocks are independent, so it suffices to study a single block. Applying the law of total variance across the two levels of randomness:

$$\mathrm{Var}[\bar{Z}_j] = \underbrace{\mathbb{E}_\gamma\left[\mathrm{Var}[\bar{Z}_j \mid \gamma_j]\right]}_{\text{avg. inner variance}} + \underbrace{\mathrm{Var}_\gamma\left[\mathbb{E}[\bar{Z}_j \mid \gamma_j]\right]}_{\text{outer PG variance}} = \frac{\sigma_A^2}{L} + \sigma_B^2,$$

where

$$\sigma_A^2 := \mathbb{E}_\gamma\left[\sigma^2_{A,\gamma}\right] \qquad \text{and} \qquad \sigma_B^2 := \mathrm{Var}_\gamma\left[\mathbb{E}_{u|\gamma}[Z \mid \gamma]\right].$$

σ_A^2 measures how much the inner Gaussian integral fluctuates for a typical PG draw. σ_B^2 measures how much the PG draw γ itself shifts the result.

Step 3: overall variance. The estimator is $\widehat{p} = \frac{1}{M}\sum_{j=1}^{M} \bar{Z}_j$ with M i.i.d. blocks, so

$$\mathrm{Var}[\widehat{p}_{\mathrm{PCPG}}(M, L)] = \frac{1}{M}\,\mathrm{Var}[\bar{Z}_j] = \frac{\sigma_A^2}{ML} + \frac{\sigma_B^2}{M}.$$

Step 4: fixed-ratio budget. Writing the total budget as $N = ML$ and the inner/outer ratio as $r = L/M$ (so $M = \sqrt{N/r}$):

$$\mathrm{Var}[\widehat{p}_{\mathrm{PCPG}}(M, L)] = \frac{\sigma_A^2}{N} + \frac{\sigma_B^2\sqrt{r}}{\sqrt{N}}.$$

The first term decays as $O(1/N)$, the same rate as MC. The second term decays only as $O(1/\sqrt{N})$, which is *slower*. For large N the second term dominates, so PCPG-MC will have higher variance than MC whenever $\sigma_B^2 > 0$.

Why σ_A^2 can be large. The problem comes from completing the square in Trick 2, which introduced the factors $e^{1/(8\gamma)}$ and $e^{iu/(2\gamma)}$ into the integrand. Both depend on $1/\gamma$ and become extreme for small γ.

To see what happens, fix z. The inner expectation is then known exactly from the HS identity,

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[e^{-i\left(z-\frac{1}{2\gamma}\right)u}\right] = \exp\left(-\tfrac{1}{2}\gamma\left(z-\tfrac{1}{2\gamma}\right)^2\right) \approx e^{-1/(8\gamma)} \quad \text{for small } \gamma,$$

which is tiny. Multiplied by $e^{1/(8\gamma)}$ it gives a value of order one. So the inner average has to produce a tiny number from terms of modulus one that almost cancel, and every bit of noise in that average is amplified by $e^{1/(8\gamma)}$. For $\gamma = 0.01$ this factor is already about $2.7 \cdot 10^5$.

In terms of variance, a single inner sample has modulus $e^{1/(8\gamma)}$, so its second moment is $e^{1/(4\gamma)}$. Near zero the density of $\mathrm{PG}(1, 0)$ decays only like $e^{-1/(8\gamma)}$, which is not fast enough to compensate, so averaged over γ the second moment is infinite: formally $\sigma_A^2 = \infty$. Small values of γ are rare, which is why the estimator still works in practice, but when they occur they produce extreme samples, and a moderately large L is needed to suppress the inner noise.

Optimal budget split. Looking at the variance formula from Step 3,

$$\mathrm{Var} = \frac{\sigma_A^2}{ML} + \frac{\sigma_B^2}{M},$$

the two terms respond to the budget differently. The first term shrinks when L is large (many inner samples per PG draw). The second term shrinks when M is large (many PG draws). Since $N = ML$ is fixed, making L larger forces M smaller and vice versa — you cannot reduce both terms simultaneously.

Substituting $M = N/L$ and minimising over L at fixed N gives the optimal split:

$$L^* = \sqrt{\frac{N\,\sigma_A^2}{\sigma_B^2}}, \qquad M^* = \sqrt{\frac{N\,\sigma_B^2}{\sigma_A^2}}, \qquad r^* := \frac{L^*}{M^*} = \frac{\sigma_A^2}{\sigma_B^2}.$$

In words: use more inner samples than outer samples if and only if $\sigma_A^2 > \sigma_B^2$. Because the inner variance is typically much larger (the $e^{1/(8\gamma)}$ blowup), the recommended ratio is $r = 4$, meaning four inner samples per PG draw. At $N = 1000$ and $r = 4$ this gives $M^* \approx 16$ outer draws and $L^* \approx 63$ inner samples per draw; equal splitting ($M = L = 32$) would leave the large inner variance insufficiently averaged.

The minimum variance at the optimal split is $2\sqrt{\sigma_A^2\sigma_B^2}/\sqrt{N}$, which is still $O(1/\sqrt{N})$. No choice of r removes the $O(1/\sqrt{N})$ scaling as long as $\sigma_B^2 > 0$.

When does σ_B^2 become small? Recall that $\sigma_B^2 = \mathrm{Var}_\gamma\Big[\mathbb{E}_{u|\gamma}[Z \mid \gamma]\Big]$: it measures how much the value of the inner integral shifts as γ changes. For a GMM with diagonal covariance, the CF of the missing features satisfies

$$\Big|\phi_{x^m|x^o}(-uw_m)\Big| \;\le\; \exp\!\left(-\frac{u^2}{2}\,D_m\right), \qquad D_m := \sum_{j\in\mathrm{mis}} w_j^2\,\sigma_{kj}^2,$$

where D_m collects the variance of the missing part of $w^T x$. When D_m is large, the CF decays very fast in u: the integrand $h(\gamma, u)$ is nearly zero unless $|u|$ is small. For any fixed γ, the relevant u values are of order $\sqrt{\gamma}$; large D_m means even moderately large γ is enough to zero out the integrand. As a result, the inner expectation $\mathbb{E}_{u|\gamma}[Z \mid \gamma]$ becomes nearly independent of γ — it is close to zero for a wide range of γ values and hardly varies — so $\sigma_B^2 \approx 0$.

Effect of missing ratio (fixed total dimension). Increasing the fraction of missing features (larger d_m, same d) adds more terms to the sum defining D_m, directly increasing it. The CF damps more strongly, σ_B^2 decreases, and PCPG-MC recovers closer to $O(1/N)$ convergence. MC behaves in the opposite way: more missing features make $w^T x$ more random, increasing σ_{MC}^2. The PCPG family therefore *gains* relative to MC as the missing ratio grows.

Effect of total dimension (fixed missing ratio). Increasing d while keeping the missing ratio $\rho = d_m/d$ constant grows both d_m and d_o proportionally. The additional terms in D_m are exactly offset by the fact that logistic-regression weights w_j shrink with larger d (the total discriminative power is spread over more features), so D_m stays roughly constant and the CF damping does not improve. At fixed missing ratio, the rate improvement for PCPG-MC described above does *not* automatically follow from adding more features; what matters is the missing-feature energy D_m, not the total dimension alone.

Comparison.

$$\mathrm{Var}[\widehat{p}_{\mathrm{MC}}(N)] = \frac{\sigma^2_{\mathrm{MC}}}{N} \;\le\; \frac{1}{4N}, \qquad \mathrm{Var}[\widehat{p}_{\mathrm{PCPG}}(M,L)] = \frac{\sigma_A^2}{N} + \frac{\sigma_B^2\sqrt{r}}{\sqrt{N}}.$$

For low-missing-rate settings where σ_B^2 is non-negligible, PCPG-MC is less sample-efficient than MC. The $O(1/\sqrt{N})$ slowdown motivates replacing the inner MC with Gauss-Hermite quadrature (PCPG-GH).

2.1.3 PCPG-GH Estimator

Variance

The PCPG-GH estimator keeps the M outer PG draws $\gamma_j \sim \mathrm{PG}(1,0)$ but replaces each inner MC average with a n_H-point Gauss-Hermite rule. Because the inner quadrature is *deterministic* given γ_j, the inner noise term σ_A^2/L vanishes entirely. The only remaining source of variance is the outer PG draw:

$$\mathrm{Var}[\widehat{p}_{\mathrm{GH}}(M)] = \frac{\sigma_B^2}{M}.$$

The total evaluation budget is $N = M \cdot n_H$ (each outer sample evaluates the CF at n_H frequencies), so

$$\mathrm{Var}[\widehat{p}_{\mathrm{GH}}(M)] = \frac{\sigma_B^2\, n_H}{N},$$

which is $O(1/N)$, matching the MC rate and eliminating the slow $O(1/\sqrt{N})$ term that limits PCPG-MC. The factor n_H is a fixed constant: with $n_H = 20$ nodes the per-budget variance is $20\,\sigma_B^2$, which is favourable whenever σ_B^2 is small compared to $\sigma^2_{\mathrm{MC}}/n_H$.

In short, PCPG-GH is a strictly better choice than PCPG-MC at the same outer budget whenever σ_A^2 is non-negligible, because replacing the inner MC with quadrature costs nothing in outer samples and eliminates the dominant noise source.

The problem does not disappear completely, however; it turns from variance into quadrature error. With $u = \sqrt{2\gamma}\, t$ the inner integrand contains the factor $\exp\!\Big(-i\left(z - \frac{1}{2\gamma}\right)\sqrt{2\gamma}\, t\Big)$, which oscillates with frequency $\left|z - \frac{1}{2\gamma}\right|\sqrt{2\gamma}$ in the Hermite variable t. This frequency is large when γ is small, but also when $|w^T x|$ is large. A fixed number of nodes cannot resolve fast oscillations, and the exact inner value $\exp(-\frac{1}{2}\gamma(z - \frac{1}{2\gamma})^2)$ it should produce is then tiny, so the rule has to sum terms of modulus one to almost zero. Any error in that sum is multiplied by $e^{1/(8\gamma)}$, which also amplifies rounding errors (in single precision, with relative accuracy $\approx 10^{-7}$, only about two digits are left at $\gamma = 0.01$). The CF damping helps when many features are missing, because it suppresses large $|u|$; with few missing features and a confident prediction, the inner quadrature error can become significant.

2.1.4 PCPG-Quad Estimator

Replacing both integrals with quadrature (Golub–Welsch for the outer PG integral, Gauss–Hermite for the inner) makes the estimator fully deterministic:

$$\mathrm{Var}[\widehat{p}_{\mathrm{Quad}}] = 0.$$

The estimator always returns the same value for the same inputs, and its output is bounded since the PG quadrature nodes γ_j are fixed positive numbers — the factor $e^{1/(8\gamma_j)}$ is a known finite constant for each node. The error is purely from quadrature approximation (bias), not variance. The amplification by $e^{1/(8\gamma)}$ is mild here, because the smallest Golub–Welsch node is not close to zero ($\gamma_{\min} \approx 0.046$ for 19 nodes, giving $e^{1/(8\gamma_{\min})} \approx 15$). The oscillation problem of the inner Gauss–Hermite rule for large $|w^T x|$ remains, since the inner rule is the same as in PCPG-GH.

2.2 Bias Analysis

2.2.1 MC Estimator

MC is unbiased for all $N \geq 1$: sampling from $p(x^m \mid x^o)$ and averaging $\sigma(w^T x)$ is a direct Monte Carlo estimate of the target expectation, so $\mathbb{E}[\widehat{p}_{\mathrm{MC}}(N)] = p(y \mid x^o)$ exactly.

2.2.2 PCPG-MC Estimator

PCPG-MC is also unbiased. Both integrals are approximated by ordinary MC, so by linearity of expectation the estimator equals the true double integral, which is $p(y \mid x^o)$ by the PG-sigmoid and Hubbard–Stratonovich identities.

2.2.3 PCPG-GH Estimator

The outer PG integral is still MC (unbiased). The inner integral is replaced by Gauss–Hermite quadrature, which introduces a small approximation error. For N_H quadrature nodes, GH integrates polynomials of degree up to $2N_H - 1$ exactly, and for smooth (analytic) integrands the error decays super-algebraically in N_H. In our setting the integrand is analytic in u, so the bias is negligible already for small N_H (in practice $N_H \geq 20$ is sufficient).

2.2.4 PCPG-Quad Estimator

Both integrals introduce a quadrature error. The Gauss–Hermite inner error is the same as for PCPG-GH. The Golub–Welsch PG quadrature with M nodes integrates polynomials in γ exactly up to degree $2M - 1$; for smooth functions of γ the error again decays rapidly with M. The maximum reliable order is $M \leq 19$ (limited by numerical precision in the moment computation). At $M = 19$ and $N_H \geq 20$, both errors are numerically negligible and the output closely matches the unbiased MC ground truth. This holds for moderate $|w^T x|$. For large positive $w^T x$ the outer error is amplified by $e^{z/2}$, which the sign flip from Section 1.2.4 removes; with the flip, the outer rule fitted directly to the sigmoid (Section 1.2.4) is more accurate than Golub–Welsch with fewer nodes. The inner Gauss–Hermite error for large $|w^T x|$ is not affected by the flip.

2.3 Comparison

Table 2.1 summarises the properties of all four estimators.

Bias. MC and PCPG-MC are exactly unbiased. PCPG-GH and PCPG-Quad introduce small quadrature errors that are negligible at the parameter choices used ($n_H \geq 20$, $M_{\mathrm{PG}} \leq 19$).

Table 2.1: Summary of all four estimators. N denotes the total evaluation budget. $r = L/M$ is the inner/outer ratio used by PCPG-MC. n_H is the number of Gauss-Hermite nodes (fixed constant). "CF" means the estimator requires evaluating $\phi_{x^m|x^o}$ rather than drawing samples from $p(x^m \mid x^o)$.

Estimator	**Variance**	**Bias**	**Rate**	**Needs**
MC	σ^2_{MC}/N	0	$O(1/N)$	samples
PCPG-MC	$\frac{\sigma_A^2}{N} + \frac{\sigma_B^2\sqrt{r}}{\sqrt{N}}$	0	$O(1/\sqrt{N})$	CF
PCPG-GH	$\sigma_B^2\, n_H/N$	$O(e^{-c\,n_H})$	$O(1/N)$	CF
PCPG-Quad	0	$O(e^{-c\,M})$	deterministic	CF

Variance and convergence rate. MC achieves $O(1/N)$ variance, bounded above by $1/(4N)$. PCPG-GH also achieves $O(1/N)$, but the effective constant is $\sigma_B^2\, n_H$: it is preferable to MC when σ_B^2 is small, which happens precisely when many features are missing and the CF damps strongly. PCPG-MC is the only estimator with a slower $O(1/\sqrt{N})$ rate; the outer PG variance σ_B^2 introduces a $\sigma_B^2\sqrt{r}/\sqrt{N}$ term that persists regardless of how the budget is split between M and L. In the high-missing-data regime $\sigma_B^2 \approx 0$ and PCPG-MC recovers $O(1/N)$, but in that regime PCPG-GH is still at least as good. PCPG-Quad has zero variance by construction; its "error" is pure bias from finite-order quadrature, which decays exponentially in the number of nodes and is negligible above $n_H = 20$ and $M_{\mathrm{PG}} = 19$.

When to use which. MC is the simplest and most reliable choice when sampling from $p(x^m \mid x^o)$ is easy and low variance is the priority. PCPG-GH is the natural first upgrade: it eliminates the inner MC noise at no extra outer-sample cost and always achieves $O(1/N)$; use it whenever evaluating the CF is cheaper than drawing samples. PCPG-MC should be preferred over PCPG-GH only when the CF is cheap but computing the Gauss-Hermite nodes is costly, or as a reference implementation to isolate σ_A^2 empirically. PCPG-Quad is the right choice when a fully deterministic, reproducible result is required — for example when differentiating through the estimator or when multiple evaluations at the same point must agree exactly.

2.4 Effect of Dimensionality

Two distinct quantities can vary: the number of missing features d_m at fixed total dimension d (equivalently, the missing ratio $\rho = d_m/d$), and the total dimension d itself at a fixed ratio ρ. Their effects are different and we treat them separately. The

key quantity that links both estimators is the *missing-feature energy*

$$D_m := \sum_{j \in \text{mis}} w_j^2 \, \sigma_{kj}^2,$$

which measures how much variance the uncertain part of $w^T x$ contributes.

Effect of the missing ratio (more missing features, fixed d). As d_m grows at fixed d, more terms enter D_m, so D_m increases.

For *MC*, the uncertain part of the dot product is $w_m^T x^m$, whose variance under $p(x^m \mid x^o)$ is exactly D_m. A larger D_m spreads the distribution of $\sigma(w^T x)$ more widely, pushing $\sigma^2_{\text{MC}} = \text{Var}[\sigma(w^T x)]$ toward its upper bound of $1/4$. *MC variance increases with the missing ratio.*

For *PCPG*, the key observation is what the CF value at a frequency actually means. The CF $\phi_X(s) = \mathbb{E}[e^{isX}]$ has modulus at most 1. Intuitively, $|e^{isX}| = 1$ for every realisation, but the complex exponential e^{isX} rotates in the complex plane as X varies; when X is widely spread, these rotations point in different directions and cancel when averaged, driving $|\phi_X(s)|$ toward zero. Concretely, if X has variance σ_X^2, then for large $|s|$ the CF decays at least as fast as $e^{-\sigma_X^2 s^2/2}$; a larger variance means faster decay.

In the PCPG integral, the CF is evaluated at the frequency $s = -uw_m$, which is proportional to the inner sample u. The relevant variance is that of the projection $w_m^T x^m$ onto the classifier direction, which equals D_m. Hence

$$\left|\phi_{x^m|x^o}(-uw_m)\right| \;\le\; \exp\!\left(-\tfrac{1}{2}u^2 \, D_m\right).$$

When D_m is large, this bound drops to nearly zero already for moderate $|u|$ — the integrand $h(\gamma, u)$ is effectively zero outside a narrow band $|u| \lesssim 1/\sqrt{D_m}$ around zero. Larger D_m makes this band narrower: "stronger CF damping" means the PCPG integrand is non-negligible only for very small u.

Two consequences follow from a narrow effective integration range. First, σ_A^2 decreases: the $e^{1/(8\gamma)}$ blowup that drives inner variance is large only when γ is small, which makes $u \sim \mathcal{N}(0, \gamma)$ small as well. In this regime the CF is near 1 regardless of D_m, so the blowup is not directly suppressed. However, for large γ — where most PG mass lies — the Gaussian $\mathcal{N}(0, \gamma)$ places u well outside the narrow band, and the CF kills those samples. The net effect is fewer extreme inner samples, lowering σ_A^2. Second, σ_B^2 decreases: the inner expected value $\mathbb{E}_{u|\gamma}[Z \mid \gamma]$ is near zero for any γ large enough that $\sqrt{\gamma} \gg 1/\sqrt{D_m}$ (essentially all large PG draws), and near $1/2$ only for the rare very small γ draws. As D_m grows, the threshold shifts to smaller γ and a larger fraction of PG draws gives $\mathbb{E}[Z|\gamma] \approx 0$, so the variation of this function over γ — which is σ_B^2 — shrinks. *PCPG variance decreases with the missing ratio.*

The same narrowing of the integrand reduces the Gauss–Hermite quadrature error for PCPG-GH: a more concentrated integrand is easier to integrate accurately with a fixed number of nodes, so the bias also decreases with ρ.

Effect of total dimension (fixed missing ratio ρ). As d grows with ρ fixed, $d_m = \rho d$ grows, but so does $d_o = (1-\rho)d$. Each individual weight scales roughly as $w_j = O(1/\sqrt{d})$ in a well-fit logistic regression (the discriminative signal is spread over more features), so each term in D_m is $O(1/d)$. Summing over $d_m = \rho d$ missing features gives $D_m = O(\rho)$: bounded, regardless of d.

For *MC*, the variance of $w_m^T x^m$ stays at $D_m = O(\rho)$, so the distribution of $\sigma(w^T x)$ approaches a fixed limit as $d \to \infty$ and σ^2_{MC} neither grows nor shrinks.

For *PCPG*, D_m stays bounded, so the CF damping does not improve and the variances σ_A^2, σ_B^2 stay roughly constant.

Higher total dimension at fixed missing ratio does not improve PCPG over MC. The relative advantage of PCPG depends on the missing ratio ρ, not on the total number of features d alone.

Summary. As the missing ratio ρ grows: MC variance increases toward $1/(4N)$, while PCPG variance (both σ_A^2 and σ_B^2) decreases due to CF concentration. As total dimension d grows at fixed ρ: both estimators are largely unaffected, since D_m stays bounded. The relative advantage of PCPG over MC is therefore tied to the missing ratio, and is strongest in the high-missing-data regime where accurate marginalisation matters most.

2.5 Effect of the Classifier's Weight Norm

The argument of Section 2.4 for a growing total dimension d assumes a well-fitted classifier, whose individual weights shrink like $O(1/\sqrt{d})$, so that $z = w^T x$ stays on the same scale. In this section we show experimentally that this assumption matters. With the default regularisation of the logistic regression, the norm of w grows with d, and PCPG with a fixed number of nodes loses accuracy at high dimensions. When the norm of w is held fixed, the effect disappears.

Setup. The data are generated with scikit-learn's `make_classification`: 10,000 samples, of which 2 features are informative, 2 are linear combinations of these, and all others are independent noise; 8,000 samples are used for training and 2,000 for testing. The density $p(x)$ is a GMM with $K = 4$ diagonal components, the classifier a logistic regression without intercept. The number of features is $d \in \{5, 16, 50, 158, 500, 1581, 5000\}$, a factor $\sqrt{10}$ apart. For every test point a fraction $\rho \in \{0.3, 0.5, 0.8\}$ of the features is hidden at random. MC and PCPG-Quad get the same budget of 1,216 evaluations: MC draws 1,216 samples, and PCPG-Quad uses $M = 19$ Golub–Welsch nodes and $n_H = 64$ Gauss–Hermite nodes. The error is the RMSE over 50 test points against the exact value, which we compute from the Gaussian mixture of $z \mid x^o$ (Section 1.1.7) with adaptive quadrature. The time is that of one batched call over all 2,000 test points on a GPU.

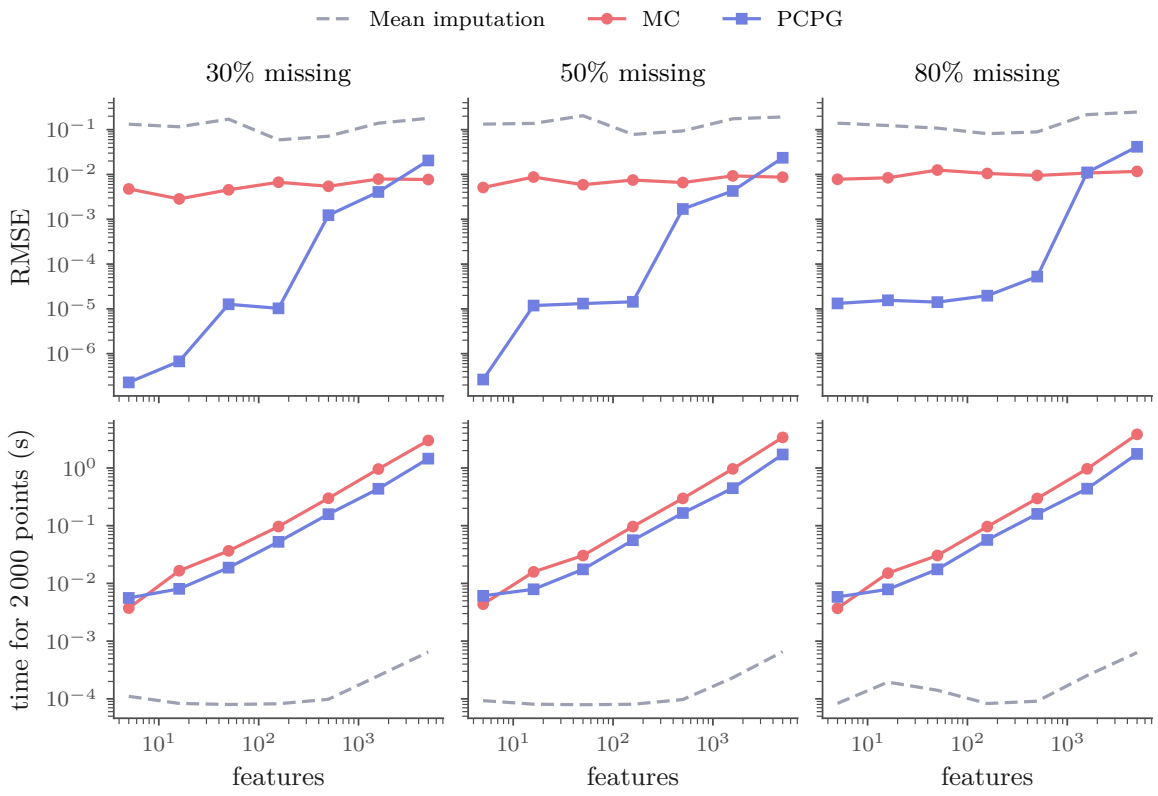


Figure 2.1: Mean imputation, MC and PCPG-Quad against the number of features d, for 30%, 50% and 80% missing features, with the default logistic regression ($C = 1$). Top: RMSE over 50 test points. Bottom: time for all 2,000 test points. MC and PCPG-Quad use a budget of 1,216 evaluations.

Observation. Figure 2.1 shows the result with scikit-learn's default regularisation ($C = 1$). Up to $d = 158$, PCPG-Quad reaches an RMSE between $2 \cdot 10^{-7}$ and $2 \cdot 10^{-5}$, two to four orders of magnitude below MC, whose error stays between $3 \cdot 10^{-3}$ and

$1.3 \cdot 10^{-2}$ at every d. From $d = 500$ on, the error of PCPG-Quad rises: to $1.2 \cdot 10^{-3}$ and $1.7 \cdot 10^{-3}$ at 30% and 50% missing for $d = 500$, to between $4 \cdot 10^{-3}$ and $1.1 \cdot 10^{-2}$ for $d = 1581$, and to between $2 \cdot 10^{-2}$ and $4 \cdot 10^{-2}$ for $d = 5000$, where it is worse than MC. The errors of MC and of mean imputation hardly depend on d.

Cause: the classifier overfits. Only four features carry information about the label, so every additional feature is noise. With $C = 1$ the logistic regression fits this noise once there are many features compared to the 8,000 training points (Table 2.2). The norm of w is about 2.2 up to $d = 158$ and grows to 2.7, 6.6 and 9.5 at $d = 500$, 1581 and 5000. At $d = 5000$ the classifier reaches 100% training accuracy, but only 80.5% test accuracy, with a test log loss of 1.22. A larger w makes $z = w^T x$ larger. At $d = 5000$, the median of $|m_k|$ over all test points and components is between 8.6 and 14.9 (depending on the missing rate), and the median standard deviation $\sqrt{v_k}$ between 2.8 and 4.8. At $d = 16$ they are between 2.4 and 2.7, and between 0.5 and 1.2.

Table 2.2: The logistic regression with the default regularisation ($C = 1$) for every number of features d: the norm of w, and accuracy and log loss on the 8,000 training and 2,000 test points.

d	$\|w\|$	**train acc.**	**test acc.**	**test log loss**
5	2.2	0.891	0.887	0.312
16	2.3	0.894	0.875	0.314
50	2.3	0.894	0.888	0.314
158	2.4	0.894	0.875	0.311
500	2.7	0.902	0.874	0.337
1581	6.6	0.938	0.817	0.764
5000	9.5	1.000	0.805	1.215

Why a larger z is harder for PCPG. Consider one component of the mixture, $z \mid k \sim \mathcal{N}(m_k, v_k)$, and substitute $u = \sqrt{2\gamma}\, t$ as in Section 1.2.2. The inner integrand of this component becomes

$$e^{1/(8\gamma)}\, e^{iat - bt^2}, \qquad a = \frac{1}{\sqrt{2\gamma}} - \sqrt{2\gamma}\, m_k, \qquad b = \gamma\, v_k,$$

and Gauss–Hermite has to approximate

$$\int_{-\infty}^{\infty} e^{-t^2}\, e^{iat - bt^2}\, \mathrm{d}t = \sqrt{\frac{\pi}{1+b}}\, \exp\!\left(-\frac{a^2}{4(1+b)}\right).$$

For large $|a|$ this value is exponentially small, but every term of the quadrature sum has a modulus of up to ω_i. The sum therefore has to cancel almost completely, which a

fixed rule can only do if its nodes resolve the oscillation e^{iat}. The n_H nodes lie within roughly $\pm\sqrt{2n_H}$ with a spacing of about $\pi/\sqrt{2n_H}$, which gives the rule of thumb $n_H \gtrsim a^2/2$. Whatever error remains is then multiplied by $e^{1/(8\gamma)}$.

Because of the flip (Section 1.2.4), m_k is mostly negative, so $a = 1/\sqrt{2\gamma} + \sqrt{2\gamma}\,|m_k|$ grows linearly in $|m_k|$, and thereby in $\|w\|$. The 19 Golub–Welsch nodes reach up to $\gamma \approx 12.8$. There, $|m_k| = 2.4$ gives $a \approx 12$, which 64 nodes still roughly resolve, while $|m_k| = 10$ gives $a \approx 50$, far more than 64 nodes can handle. The largest nodes carry small weights, so the rule of thumb is only a rough guide, but it explains the trend. The second parameter b grows with $\|w\|^2$ and makes the integrand narrower, which also needs more nodes.

Two measurements support this. At $d = 500$ and 30% missing, doubling the inner nodes to $n_H = 128$ reduces the error from $1.2 \cdot 10^{-3}$ to $1.5 \cdot 10^{-5}$, so there the inner rule is the limit. At $d = 5000$, neither $n_H = 256$ nor the maximal $M = 19$ outer nodes bring the error below about $7 \cdot 10^{-3}$: the scale of z exceeds what both rules cover.

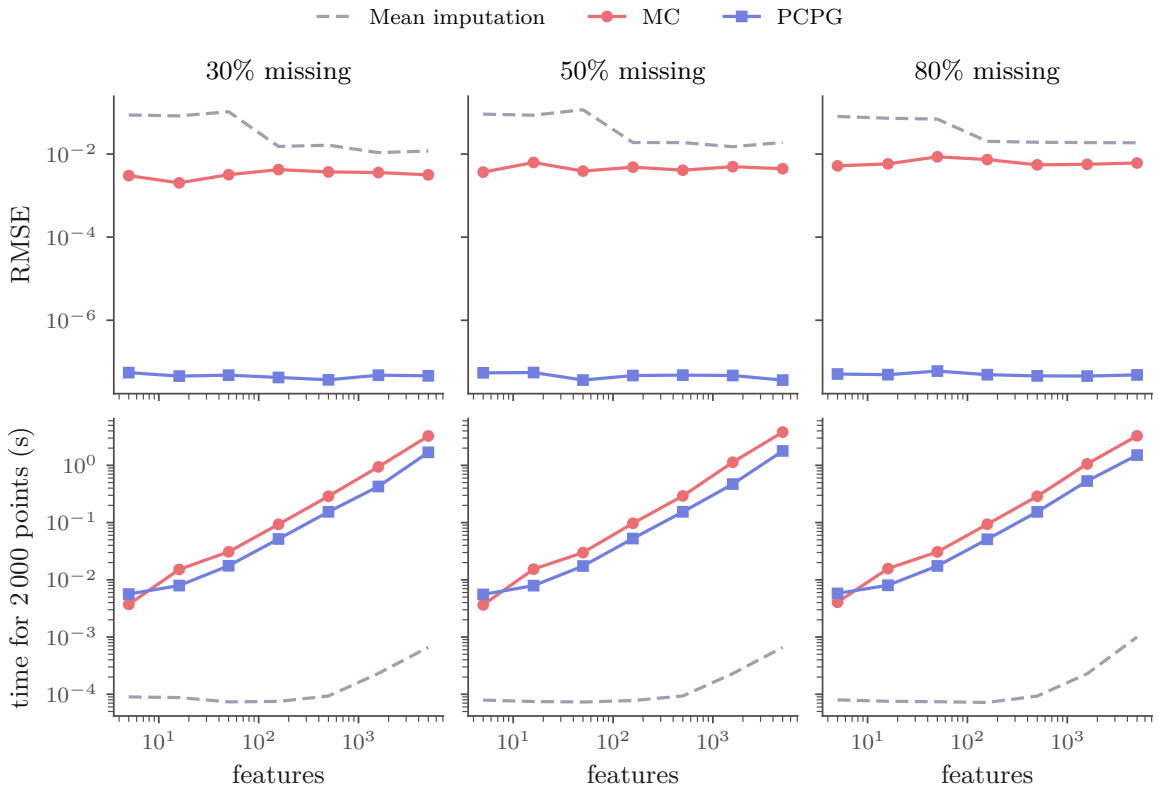


Figure 2.2: The same comparison as in Figure 2.1, with the weights of the logistic regression rescaled to unit norm, $w/\|w\|$, for every number of features d.

Holding the norm fixed. To separate the effect of d from that of $\|w\|$, we rescale the weights of the default classifier to unit norm, $w \leftarrow w/\|w\|$, for every d. This keeps

the direction of w, and with it every decision of the classifier, and only puts $z = w^T x$ on the same scale for every d.

With the norm held fixed (Figure 2.2), the error of PCPG-Quad no longer depends on d. It lies between $3.6 \cdot 10^{-8}$ and $6 \cdot 10^{-8}$ for every d and missing rate, at the precision of the single precision floating point numbers used in the computation. This includes $d = 1581$ and $d = 5000$, where the same classifier with its original norm gave errors of up to $4 \cdot 10^{-2}$. MC stays between $2 \cdot 10^{-3}$ and $9 \cdot 10^{-3}$. The scale of z thus explains the whole loss of accuracy at high dimensions. Since the unit norm is smaller than the norm of about 2.2 that the default classifier has at low dimensions, PCPG-Quad is also more accurate here than in Figure 2.1 at small d.

Rescaling is a controlled experiment, not a way to fix the classifier, since it keeps the overfitted direction of w. In practice, stronger regularisation keeps the norm small and also improves the classifier: with C chosen such that $\|w\| = 2.2$ at $d = 5000$, the test accuracy rises from 80.5% to 83.6% and the test log loss falls from 1.22 to 0.40.

Runtime. The bottom rows of both figures are the same, since the time does not depend on w. For all methods it grows linearly in d. At the same budget, PCPG-Quad evaluates the characteristic function at 1,216 frequencies per test point and MC draws 1,216 samples. From $d = 16$ on, PCPG-Quad is about twice as fast as MC in our implementation, for example about 1.4 s against 3.0 s at $d = 5000$, mostly because MC draws all d features of every sample, including the observed ones.

Summary. The accuracy of PCPG-Quad with a fixed number of nodes depends on the scale of $z = w^T x$, that is on m_k and v_k relative to the quadrature rules, and not on the number of features as such. The argument of Section 2.4 holds for a regularised classifier. An overfitted classifier makes z large, and then more nodes are needed than a rule chosen for small z provides. In practice, the classifier should be regularised, which also improves it. Alternatively, the number of inner nodes can be chosen per test point from the largest $|m_k|$ and v_k.

2.6 Special Case: Closed Form for GMMs

For a fixed γ, the integral over u together with the expectation over x^m only undoes the HS step: they compute

$$g(\gamma) := \mathbb{E}_{p(x^m \mid x^o)}\left[\exp\left(\tfrac{1}{2}z - \tfrac{1}{2}\gamma z^2\right)\right], \qquad p(y \mid x^o) = \tfrac{1}{2}\,\mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)}[g(\gamma)]\,.$$

HS is needed because, for a general density, $g(\gamma)$ cannot be computed directly. For a GMM with diagonal covariances it can. Within component k, $z = w_o^T x^o + w_m^T x^m$ is a constant plus a weighted sum of independent Gaussians, so

$$z \mid k \sim \mathcal{N}(m_k, v_k), \qquad m_k = w_o^T x^o + w_m^T \mu_k^m, \qquad v_k = \sum_{d \in \text{mis}} w_d^2 \, \sigma_{k,d}^2.$$

The expectation of $\exp(\frac{1}{2}z - \frac{1}{2}\gamma z^2)$ under a Gaussian is a Gaussian integral (both exponents are quadratic in z), and completing the square in z gives

$$g(\gamma) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \frac{1}{\sqrt{1+\gamma v_k}} \exp\left(\frac{m_k + \frac{1}{4} v_k - \gamma m_k^2}{2\,(1+\gamma v_k)} \right).$$

For $v_k = 0$ (nothing missing) this reduces to the PG identity itself, as it should. Only the outer integral over γ remains, and it can be handled with MC or the Golub–Welsch rule as before. The integrand is real, positive and bounded, so the small-γ problems discussed above do not occur, and each value of γ costs K operations instead of $K \cdot n_H$.

This does not carry over to general probabilistic circuits. A PC with Gaussian leaves is also a mixture of Gaussians, but with exponentially many components (induced trees) that cannot be enumerated. The circuit can only compute expectations of functions that factorise over the features, and $g(\gamma)$ does not, because of the γz^2 term. After the HS step the exponent is linear in z and factorises, which is exactly why PCPG needs the characteristic function. For GMMs the closed form is therefore mainly useful as an exact reference: it isolates the error introduced by the inner quadrature of PCPG.

2.7 Training with Missing Features

So far the classifier w was trained on complete data, and features were only missing at test time. In practice, the training data are often incomplete as well. In this section we show that the expectation $\mathbb{E}_{p(x^m \mid x^o)}[\sigma(w^T x)]$ then also appears in training, and that PCPG can be used inside the training loss. We compare this with the usual ways of training a logistic regression on incomplete data.

2.7.1 Training in Two Stages

Stage 1: the density. The density model $p(x)$ is fit first, on the incomplete training points, by maximising the likelihood of the observed features,

$$\log p(x^o) = \log \int p(x^o, x^m) \, \mathrm{d}x^m.$$

For a GMM and for a PC this marginal is tractable: the missing features are integrated out by leaving out their factors (for a PC, by setting their leaves to one). This stage therefore needs neither PCPG nor sampling. Afterwards $p(x)$ is kept fixed.

Stage 2: the classifier. The classifier is fit to the prediction that is made at test time. For a training point with observed features x_n^o and label y_n, this prediction is the expected prediction

$$p(y_n = 1 \mid x_n^o) = \mathbb{E}_{p(x^m \mid x_n^o)} \left[\sigma(w^T x)\right], \qquad p(y_n = 0 \mid x_n^o) = 1 - p(y_n = 1 \mid x_n^o),$$

and w minimises the negative log-likelihood of the labels under it, the *marginal likelihood* loss,

$$J(w) = -\frac{1}{n} \sum_{n'=1}^{n} \log p(y_{n'} \mid x_{n'}^o) + \frac{1}{2Cn} \|w\|^2. \tag{2.1}$$

The second term is the L_2 penalty of scikit-learn's logistic regression, scaled to one training point, with $C = 1$. It keeps w finite and makes the optimum unique. Without missing features, J is exactly the objective of an ordinary logistic regression.

The expectation inside the loss. The loss contains the expectation for every training point, and it has to be evaluated in every step of the optimisation, together with its gradient. With PCPG, the expectation is

$$p(y = 1 \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \sum_{j=1}^{M} w_j \sum_{i=1}^{N} \omega_i \exp\left(\frac{1}{8\gamma_j} + i u_{ij} \left(\frac{1}{2\gamma_j} - w_o^T x^o\right)\right) \phi_{x^m \mid x^o}(-u_{ij}\, w_m), \qquad u_{ij} = \sqrt{2\gamma_j}\, t_i,$$

(Section 1.2.3). The classifier w enters it only through the phase $w_o^T x^o$ and through the frequencies $-u_{ij} w_m$ at which the characteristic function is evaluated. For a GMM the characteristic function is a smooth closed-form expression in these frequencies (Section 1.2.1), and for a PC it is computed by one pass through the circuit. The estimate is therefore a smooth function of w, and its gradient is obtained by automatic differentiation. Since the nodes are fixed, the estimate is also deterministic: every evaluation of the loss at the same w gives the same value. The loss is the exact marginal likelihood up to the quadrature error of PCPG.

Monte Carlo inside the loss. The obvious alternative is to estimate the expectation by sampling,

$$\hat{p} = \frac{1}{S} \sum_{s=1}^{S} \sigma\left(w^T x^{(s)}\right), \qquad x^{m,(s)} \sim p(x^m \mid x^o),$$

with new samples in every step. The samples do not depend on w, so the gradient is simply the average of the gradients at the samples. The estimate $\hat{p}$ itself is unbiased,

but the loss is not: since log is concave, Jensen's inequality gives

$$\mathbb{E}\left[-\log \hat{p}\right] \geq -\log \mathbb{E}[\hat{p}] = -\log p,$$

so the Monte Carlo loss is too large on average. A second-order expansion of log around p gives the size of this bias,

$$\mathbb{E}\left[-\log \hat{p}\right] + \log p \approx \frac{\mathrm{Var}[\hat{p}]}{2p^2} = \frac{\mathrm{Var}[\sigma(z)]}{2Sp^2},$$

where $\mathrm{Var}[\sigma(z)]$ is the variance of the prediction over the missing features. The bias decreases like $1/S$, but it does not average out over the steps of the optimisation. It acts as an additional penalty on how much the prediction depends on the missing features, and it is minimised by making w smaller. The Monte Carlo objective therefore has its optimum at a smaller w than the marginal likelihood. For $S = 1$ the loss is the average loss over a single random completion of every point, $\mathbb{E}[-\log \sigma(\pm w^T x)]$, which is the objective of training on randomly imputed data (multiple imputation). With growing S, the Monte Carlo objective moves from this objective to the marginal likelihood.

2.7.2 Compared Models

All models are logistic regressions without intercept. They differ only in the training points they see and in the prediction their loss is built from (Table 2.3):

- **Complete data:** an ordinary logistic regression on the complete training points. This is an oracle, since these points are not available when features are missing. It shows what training could reach without missing features.
- **Mean imputation:** an ordinary logistic regression on the training points with every missing feature replaced by the mean of its observed values.
- **MC, PCPG, Exact:** the marginal likelihood (2.1) on the incomplete training points, with the expectation estimated by Monte Carlo with S samples, by deterministic PCPG with $M \times N$ nodes, or computed exactly. Exact uses the direct route for Gaussian mixtures (Section 1.1.7) with Gauss–Hermite quadrature for every component. It is only available for the GMM, and serves as the target of MC and PCPG.

MC, PCPG and Exact are trained in the same way, with Adam on minibatches, starting from $w = 0$. Complete data and mean imputation are ordinary logistic regressions and are fit with scikit-learn, which finds their optimum exactly.

Model	Training points	Prediction in the loss
Complete data	complete x	$\sigma(w^T x)$
Mean imputation	x^o, x^m filled by means	$\sigma(w^T x)$
MC	x^o	$\frac{1}{S}\sum_s \sigma(w^T x^{(s)})$
PCPG	x^o	PCPG-Quad, $M \times N$ nodes
Exact	x^o	$\sum_k \tilde{\pi}_k(x^o)\mathbb{E}_{\mathcal{N}(m_k,v_k)}[\sigma(z)]$

Table 2.3: The compared models. The last three minimise the marginal likelihood (2.1), the first two an ordinary logistic regression loss. All use the same L_2 penalty.

2.7.3 Evaluation

Every trained w is scored in three ways.

- **Objective gap** $J(w) - J(w^*)$, where J is the exact marginal likelihood (2.1) on the training points and w^* its optimum, found by full-batch L-BFGS. It measures how well a model minimises the marginal likelihood, so its target is Exact. For complete data and mean imputation it also contains the difference between their objective and the marginal likelihood.
- **Test accuracy on incomplete points:** every model predicts with the exact expected prediction on test points with the same fraction of missing features, so the models only differ in w.
- **Test accuracy on complete points:** the accuracy of $\sigma(w^T x)$ on the complete test points, i.e. how good w is as a classifier.

For both accuracies the target is the complete-data model, the classifier that training could reach if no features were missing.

2.7.4 Setup

The data are again generated with scikit-learn's `make_classification`, with 10,000 samples, 8,000 for training and 2,000 for testing. In contrast to Section 2.5, half of the features are informative and the others are noise, without redundant features: with only two informative features, almost all missing features would be irrelevant for the label. The number of features is $d \in \{5, 10, 20, 50, 100\}$. In every training and test point, $\lfloor \rho d \rfloor$ features are hidden at random, with $\rho \in \{0.3, 0.5, 0.8\}$. The density is a GMM with $K = 4$ diagonal components, fit on the incomplete training points. PCPG uses $M = 10$ Golub–Welsch nodes and $N = 32$ Gauss–Hermite nodes, and MC gets the same budget of $S = 320$ samples per point and step. Exact uses 64 Gauss–Hermite nodes per component. Adam runs for 100 epochs with minibatches of 1,024 points and a learning rate of 0.01.

2.7.5 Results

All results are for a single random choice of the hidden features per setting. With 2,000 test points, an accuracy has a standard error of about 0.9 percentage points, so smaller differences in accuracy are not meaningful.

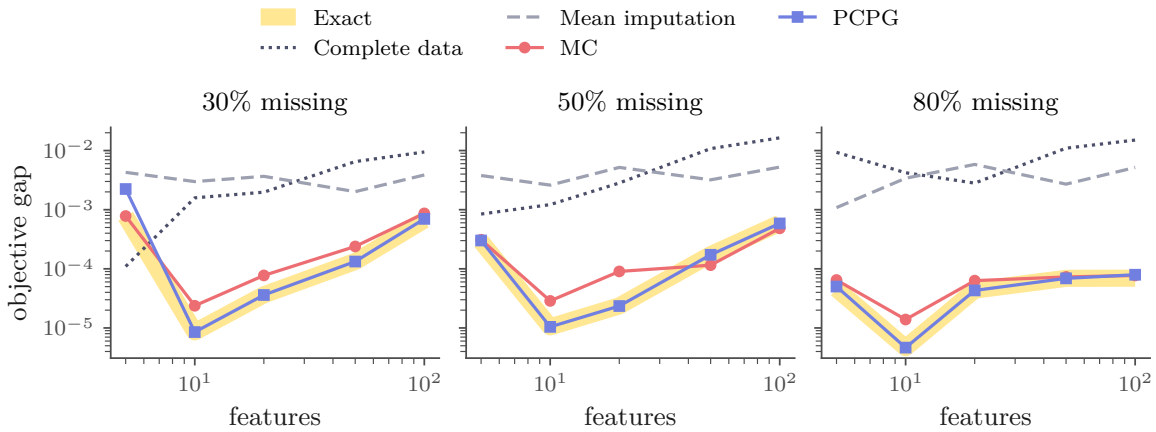


Figure 2.3: Objective gap $J(w) - J(w^*)$ of the trained classifiers against the number of features d, for 30%, 50% and 80% missing features. J is the exact marginal likelihood (2.1) on the training points, and the target is the classifier trained with the exact expectation (yellow band). MC uses $S = 320$ samples, PCPG 10×32 nodes.

The marginal likelihood is reached. Figure 2.3 shows how well every model minimises the marginal likelihood. MC, PCPG and Exact mostly end between $5 \cdot 10^{-6}$ and 10^{-4} above the optimum, complete data and mean imputation mostly between 10^{-3} and $2 \cdot 10^{-2}$. The baselines minimise a different objective, so their gap does not shrink with more training: it is the price of ignoring the uncertainty of the missing features. PCPG follows Exact closely in almost all settings, i.e. its quadrature error does not affect the training. MC lies slightly above it, by up to $7 \cdot 10^{-5}$ for 10 and 20 features. With $S = 320$ samples its bias $\mathrm{Var}[\sigma(z)]/(2Sp^2)$ is small, but visible. At both ends of the range of d the gaps are larger, for all three methods including Exact, so the cause is the optimiser and not the expectation: Adam with a fixed learning rate does not settle exactly at the optimum. At 5 features it has to reach large weights ($\|w^*\| \approx 3$), and Exact stops $7.8 \cdot 10^{-4}$ above the optimum at 30% missing, more than the complete-data classifier ($1.1 \cdot 10^{-4}$). At 100 features there are many weights, and all three stop at about $6 \cdot 10^{-4}$ for 30% and 50% missing. In one setting PCPG falls behind Exact: with 5 features and 30% missing only a single feature is hidden, so $z \mid x^o$ is concentrated and the characteristic function hardly decays, which is the hardest case for the fixed nodes of PCPG. Its gap of $2.2 \cdot 10^{-3}$ is three times that of Exact.

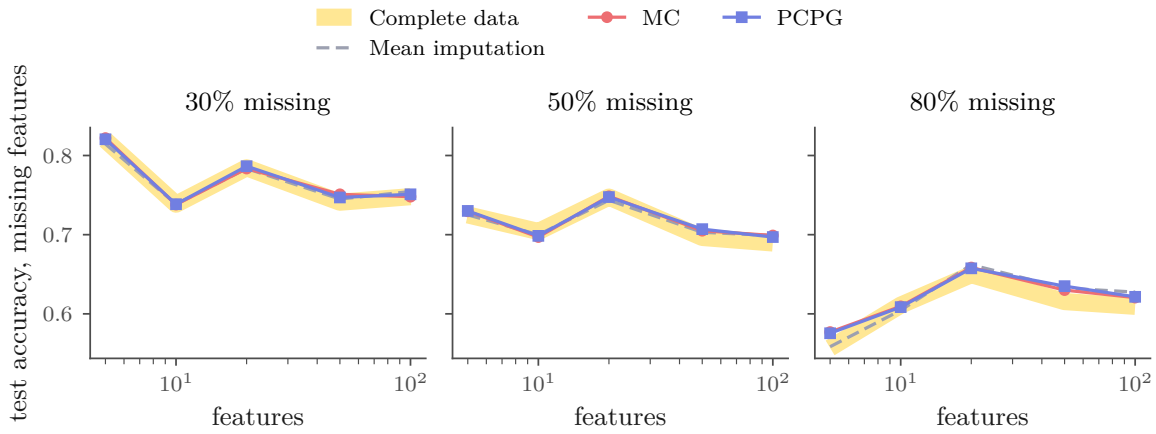


Figure 2.4: Test accuracy on test points with the same fraction of missing features, every classifier with the exact expected prediction. The target is the classifier trained on complete data (yellow band).

Accuracy on incomplete test points. When the test points are incomplete as well (Figure 2.4), all classifiers reach almost the same accuracy at 30% and 50% missing. At 80% missing, the classifiers trained on the marginal likelihood are more accurate than the one trained on complete data, by 2.0 percentage points at 5 features (57.7% against 55.7%), 1.9 at 50 features and 1.4 at 100 features. The complete-data classifier is optimal for complete points, not for the expected prediction it is used with here, while the marginal likelihood trains exactly for that prediction. Mean imputation lies close to the marginal likelihood at 50 and 100 features, but 1.9 percentage points below it at 5 features.

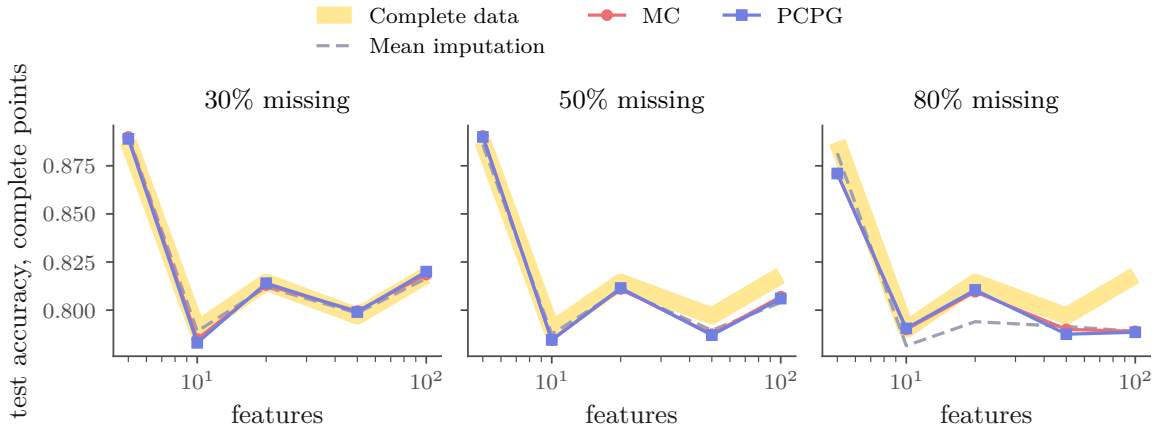


Figure 2.5: Test accuracy of $\sigma(w^T x)$ on the complete test points, i.e. how good the classifier is when nothing is missing. The target is the classifier trained on complete data (yellow band).

Accuracy on complete test points. Figure 2.5 shows how much of the classifier is lost by training on incomplete data, measured on complete test points. At 30% and 50% missing, all classifiers lie within 1.5 percentage points of the complete-data classifier. At 80% missing they fall behind it, by up to 2.9 percentage points at 100 features. Here, the classifiers trained on the marginal likelihood are ahead of mean imputation at 10 and 20 features (81.1% against 79.4% at 20 features), and level with it at 50 and 100 features. At 5 features, mean imputation is ahead by 1.1 percentage points.

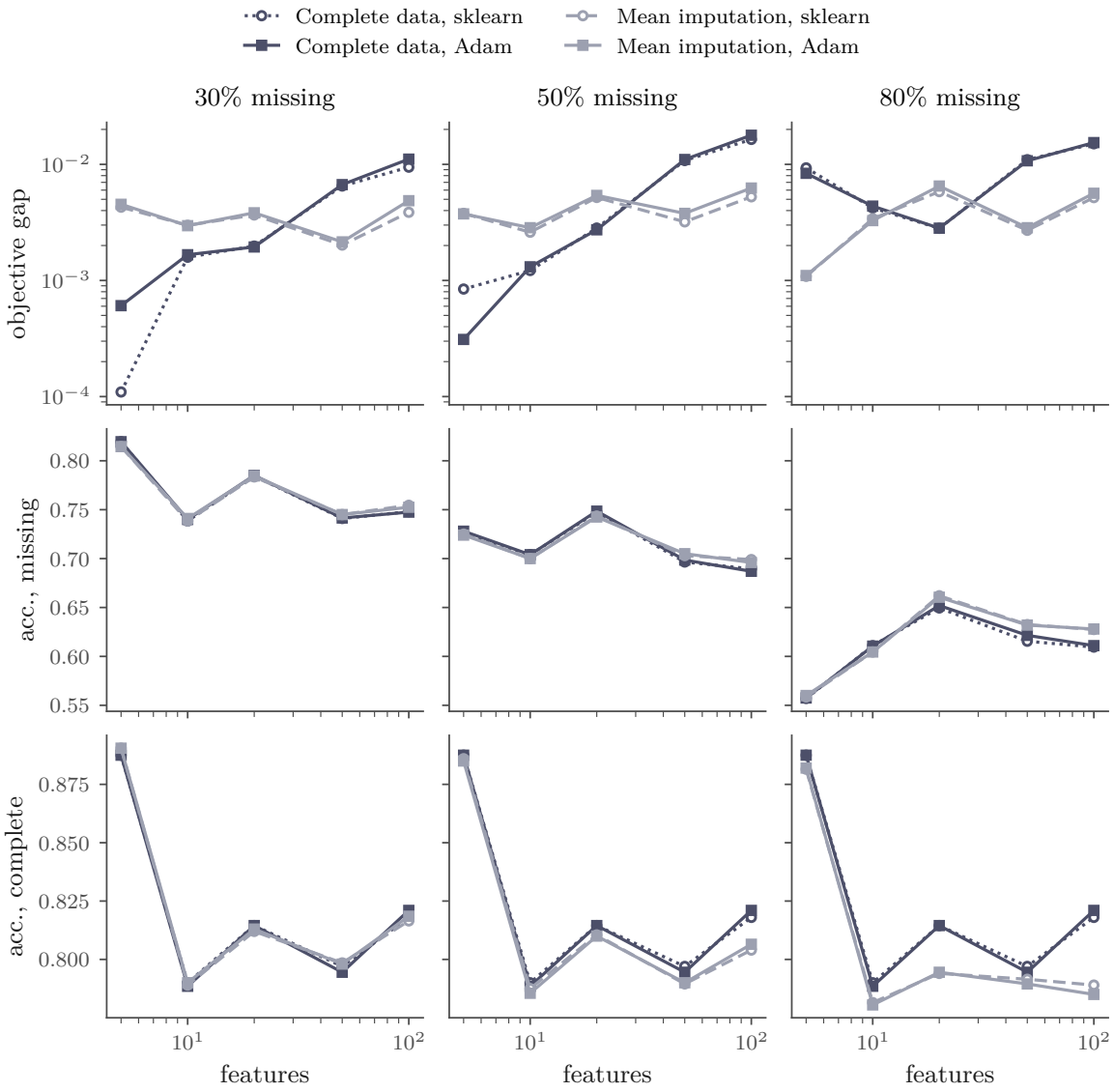


Figure 2.6: The two ordinary logistic regressions, fit with scikit-learn (open markers) and with the same Adam schedule as the other models (filled markers): objective gap, and test accuracy on incomplete and on complete test points.

Adam against an exact solver. To check that the training by Adam does not distort the comparison, we also fit complete data and mean imputation with Adam, in the same way as MC, PCPG and Exact (Figure 2.6). Both fits give practically the same accuracies, and the same objective gap except at 5 features, where Adam does not reach the optimum of the logistic regression. There its gap differs from that of scikit-learn by up to a factor of five, in both directions, since the gap is measured on the marginal likelihood and not on the objective the two fits minimise. This is the same limitation of the optimiser as for Exact above, and it does not change the comparison between the models.

Summary. PCPG makes the marginal likelihood usable as a training loss: the training with PCPG reaches the same classifier as training with the exact expectation, without sampling and without the systematic bias of Monte Carlo. Training on the marginal likelihood makes the classifier better suited to incomplete test points than training on complete data, with gains of up to two percentage points of accuracy at 80% missing features. On complete test points, it loses less than mean imputation at high missing rates and moderate dimensions. In terms of accuracy, the differences between the methods are small on this data, while in terms of the objective they are clear: only the marginal likelihood models reach its optimum.

2.8 Related Work

Here, we just focus on works that also try to calculate

$$\mathbb{E}_{P(x^m|x^o)}\left[\sigma(w^T x)\right]$$

but use a different approach.

2.8.1 Naive Conformant Learning (NaCL)

Khosravi et al. [5] first point out that this expectation is hard to compute in general: even with a naive Bayes feature distribution, it is NP-hard. They also note that the popular shortcut of mean imputation only matches the expectation when the classifier is linear *and* the features are independent — for the sigmoid it is always biased.

Their fix is to pick a feature distribution on which the expectation *is* cheap, namely a naive Bayes model over binary features. The trick is that naive Bayes and logistic regression are dual to each other: every naive Bayes induces a logistic regression, and many naive Bayes models induce the same one. They call a naive Bayes *conformant*

with the given LR if both give the same class probabilities on fully observed inputs, and learn such a distribution via geometric programming ("naive conformant learning"). Once this naive Bayes is fitted, the expected prediction is just a marginal inference query and runs in linear time.

Difference to us: they keep the feature distribution simple (naive Bayes on binary inputs) so the expectation is exact and cheap; we keep the feature distribution rich (a GMM or PC on continuous inputs) and instead use a stochastic trick to handle the sigmoid. They also have to refit the feature distribution whenever the classifier changes, while ours is trained once and reused.

2.8.2 Joint Modelling via SAEM

Jiang, Josse, and Lavielle [6] assume a joint Gaussian model on the covariates, $x \sim \mathcal{N}(\mu, \Sigma)$, together with logistic regression on top. Their main contribution is on the training side: they fit (μ, Σ, β) jointly from training data that may itself contain missing values, using a stochastic-approximation EM algorithm with Metropolis–Hastings updates.

For prediction at test time, they simply do plain Monte Carlo: sample $x^{m,(s)} \sim p(x^m \mid x^o)$ from the fitted Gaussian and average,

$$p(y \mid x^o) \approx \frac{1}{S} \sum_{s=1}^{S} p\Big(y \mid x^o, x^{m,(s)}\Big)$$

This is exactly the MC baseline we compare against, just with a single Gaussian instead of a mixture as the density.

Difference to us: they sample the missing features and average the sigmoid; we never sample the missing features, but instead rewrite the sigmoid so that only the characteristic function of $p(x^m \mid x^o)$ is needed. Their feature distribution is also limited to a single Gaussian, while ours can capture multimodal shapes.

Appendix

Bibliography

[1] Nicholas G Polson, James G Scott, and Jesse Windle. "Bayesian inference for logistic models using Pólya–Gamma latent variables." In: *Journal of the American statistical Association* 108.504 (2013), pp. 1339–1349 (cit. on p. 1).

[2] R. L. Stratonovich. "On a method of calculating quantum distribution functions." In: *Soviet Physics Doklady* 2 (1957), pp. 416–419 (cit. on p. 2).

[3] J. Hubbard. "Calculation of partition functions." In: *Physical Review Letters* 3.2 (1959), pp. 77–78. DOI: 10.1103/PhysRevLett.3.77 (cit. on p. 2).

[4] Gene H. Golub and John H. Welsch. "Calculation of Gauss Quadrature Rules." In: *Mathematics of Computation* 23.106 (1969), pp. 221–230 (cit. on p. 12).

[5] Pasha Khosravi et al. "What to expect of classifiers? Reasoning about logistic regression with missing features." In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. 2019. URL: https://arxiv.org/abs/1903.01620 (cit. on p. 40).

[6] Wei Jiang, Julie Josse, and Marc Lavielle. "Logistic regression with missing covariates—Parameter estimation, model selection and prediction within a joint-modeling framework." In: *Computational Statistics & Data Analysis* 145 (2020), p. 106907. URL: https://arxiv.org/abs/1805.04602 (cit. on p. 41).